

RL for Active Beacon Localization

Master Thesis

July 31, 2026

Presented by:
Edoardo Tommasi
4489925

First Reviewer:
RA Jesus E. Aleman G.

Second Reviewers:
PD Dr.-Ing. Sven Wachsmuth
Prof. Simone Martini

*We are nothing but agents of failure,
and through failure we reach heights
that we thought we'd never dared
to soar.*

Abstract

This thesis investigates autonomous indoor navigation using **Range-Only Single-Beacon (ROSB)** signals by mixing **deep reinforcement learning (RL)** controllers with dedicated **state estimation backends**. To tackle the inherent observability constraints of range-only, the problem is formulated across **Partially Observable Markov Decision Processes (POMDPs)**, **Hidden Markov Models (HMMs)**, and standard **MDP RL** frameworks. Training and evaluation are conducted within a custom Gymnasium-compliant environment built upon the two-dimensional **IR-Sim** engine, leveraging diverse layout topologies from the **HouseExpo dataset**. The autonomous agent is tasked with estimating an unobserved target’s location, navigating to its proximity, and docking safely without colliding with obstacles.

Utilizing **Stable-Baselines3**, we benchmark a range of actor-critic algorithms, including off-policy paradigms (**SAC**, **TD3**, **DDPG**) and on-policy baselines (**PPO**). The navigation pipeline combines three-sample linear **Least Squares (LS)** multilateration and non-parametric **Particle Filters (PF)** with **A* path planning**, governed by a high-level **finite-state machine** that orchestrates search, approach, and terminal docking phases.

Experimental results confirm the system’s effectiveness, led by **SAC** operating with **Least Squares** and then **Particle Filter (LS-PF)** and **TD3** with **PF**, attaining success rates of **91.9%**, **88.0%**, and **86.9%**, respectively. Crucially, the analysis uncovers a tight coupling between **controller motion profiles** and **estimator reliability**. While curvilinear trajectories maintain spatial baseline variation necessary for accurate 3-point LS estimations, straight-line trajectories or localized stalling collapse the multilateration matrix, inducing severe **ill-conditioning** and **filter divergence**. Furthermore, policy architecture heavily dictates noise tolerance: maximum-entropy exploration (**SAC**) effectively mitigates observation noise to leverage small LS batches, whereas on-policy updates (**PPO**) suffer catastrophic **rollout corruption**. Deterministic and standard continuous methods (**DDPG** and, to a lesser degree, **TD3**) also exhibit heightened sensitivity to unfiltered estimation noise. These insights emphasize that successful ROSB deployment hinges on **co-designing** noise-resilient, maximum-entropy control schemes alongside robust, non-parametric state estimators.

Contents

Abstract	i
List of Figures	iv
List of Tables	v
1 Introduction	3
2 Related Work	5
2.1 Evolution of ROSB Estimation Techniques	5
2.2 Classical Active Perception	7
2.3 Learning-Based Active Guidance	8
2.4 Marine-to-Indoor	8
3 Theoretical Framework	9
3.1 Hidden Markov Models	9
3.1.1 Target Tracking Formulation	9
3.1.2 Least Squares Estimation	10
3.1.3 Particle Filtering	11
3.2 Reinforcement Learning	13
3.2.1 Foundations and The MDP Framework	14
3.2.2 The Algorithmic Interaction Loop	14
3.2.3 Policies and Value Functions	15
3.2.4 Taxonomy of Reinforcement Learning Approaches	17
4 Methodology	19
4.1 Development and Architecture	19
4.1.1 Map Parsing and Occupancy Grid Generation	20
4.1.2 Gymnasium Environment Encapsulation	22
4.1.3 Estimation Backend	22
4.1.4 Reinforcement Learning Framework	23
4.1.5 Path Planning and Waypoint Pruning	26
4.2 Integration and Execution Framework	28
4.2.1 Vectorized Environments	29

4.2.2	Data Scaling	30
4.2.3	Data Recording	31
4.2.4	Crash Protection	31
4.2.5	Evaluation Protocol	32
5	Evaluations and Comparisons	33
5.1	Experimental Settings and Hyperparameters	33
5.2	Evaluation Protocol	34
5.3	Training Analysis	35
5.3.1	Termination Criteria Analysis	35
5.3.2	Reward Analysis	40
5.3.3	Timesteps Analysis	42
5.3.4	Estimation Error Analysis	44
5.4	Results	46
5.4.1	SAC	46
5.4.2	TD3	47
5.4.3	DDPG	48
5.4.4	PPO	49
5.4.5	Evaluation Summary	50
5.5	Conclusion	52
	Bibliography	54
	Acknowledgements	57

List of Figures

2.1	The hollow ring of uncertainty in an IR-Sim environment	6
3.1	Diagram showing the functioning of a Least Squares algorithm.	11
3.2	Diagram showing the functioning of a Particle Filter algorithm. Credits: (Mikoff, 2020)	12
3.3	Diagram of the sequential Reinforcement Learning interaction loop. Credits: (Simonini & The Hugging Face Team, 2022)	15
4.1	System architecture mapping the offline data pipeline and the environment interaction loop.	20
4.2	Example of the map conversion process.	20
4.3	The Reinforcement Learning architecture.	24
4.4	Sequential progression of the particle-based goal generation and naviga- tion framework across four different steps.	29
5.1	Comparison of performance metrics across 3×10^6 timesteps for all policy-estimator combinations.	35
5.2	Full mean reward across 3×10^6 timesteps for all policy-estimator combi- nations.	40
5.3	Zoomed mean reward across 3×10^6 timesteps for all policy-estimator combinations.	40
5.4	Mean episode length across 3×10^6 timesteps for all policy-estimator combinations.	42
5.5	Full estimation error trajectories across 3×10^6 timesteps for all policy- estimator combinations.	44
5.6	Zoomed estimation error performance ($y \in [0, 1.0]$).	44

List of Tables

5.1	Standardized Reinforcement Learning hyperparameters.	33
5.2	Quantitative performance metrics for SAC with Least Squares estimation backend.	46
5.3	Quantitative performance metrics for SAC with Particle Filter estimation backend.	46
5.4	Quantitative performance metrics for TD3 with Least Squares estimation backend.	47
5.5	Quantitative performance metrics for TD3 with Particle Filter estimation backend.	47
5.6	Quantitative performance metrics for DDPG with Least Squares estimation backend.	48
5.7	Quantitative performance metrics for DDPG with Particle Filter estimation backend.	48
5.8	Quantitative performance metrics for PPO with Least Squares estimation backend.	49
5.9	Quantitative performance metrics for PPO with Particle Filter estimation backend.	49
5.10	Consolidated results in new environments across policy and estimator configurations.	50

1

Introduction

As autonomous mobile robots increasingly permeate daily human environments, their ability to operate reliably in complex, unstructured indoor spaces will become essential. A fundamental challenge in these deployments is navigation under spatial uncertainty. In many real-world domestic and industrial scenarios, a robot may not have access to the exact spatial coordinates of its destination. Instead, it may need to navigate using limited sensory input, such as strictly continuous distance measurements to a target. This scenario, formally known as the Range-Based Active Beacon Localization problem (Masmitja et al., 2019, 2022), requires the robot to simultaneously estimate the target’s location and safely navigate towards it without colliding with dynamic or static obstacles.

The primary objective of this work (Tommasi, 2026) is to provide a robust solution to this uncertainty problem by developing a Deep Reinforcement Learning (RL) (Sutton & Barto, 2018; Muel & Kavlakoglu, n.d.) framework capable of guiding a robot through complex layout scenarios. To simulate realistic domestic environments, this work utilizes two-dimensional maps derived from the HouseExpo (Li et al., 2020) dataset within the IR-Sim (Han et al., 2026) simulation environment. In this controlled setup, the robot leverages range-only data to accurately localize an active beacon, calculate a safe and efficient trajectory, and ultimately halt within a predefined threshold distance of the target.

Central to achieving this objective is the paradigm of *Active Perception* (Bajcsy, 1988; Bajcsy et al., 2018). While conventional robotic systems often treat sensing and action as completely separate pipelines, range-only navigation requires a tight coupling between the two. Because a single distance measurement provides only a radius of possibilities rather than a precise coordinate, a robot cannot pinpoint a target by standing still or driving in a straight line. Instead, it must execute purposeful, non-linear maneuvers to gather diverse spatial perspectives. Within this framework, the deep reinforcement learning policy serves as the operational engine for active perception. By interacting dynamically with the environment, the RL agents learn to automatically balance goal-directed navigation with strategic exploration, exploiting physical motion as a primary mechanism for information gathering.

To implement this active perception framework, the proposed system integrates deep reinforcement learning with probabilistic localization and deterministic pathfinding. Using the Stable Baselines3 (Raffin et al., 2021) library, several state-of-the-art RL algorithms (specifically TD3, SAC, DDPG, and PPO) are implemented to govern the robot’s exploration and decision-making processes. Parallel to the learning agents, Particle Filter (Setoodeh et al., 2022) and Least Squares (Li, 2025) estimation methods are employed to dynamically calculate the beacon’s precise coordinates. An A* (Foad et al., 2021) pathfinding algorithm is utilized to generate an optimal trajectory to the estimated location. Furthermore, the robot’s overall behavior is managed through a state machine (Sipser, 2012) architecture, dividing the navigation task into distinct

searching, following, and docking phases. By systematically analyzing and comparing these methodologies, this work demonstrates the efficacy of combining modern RL techniques with classical estimation algorithms to resolve complex spatial uncertainty in robotics.

The remainder of this thesis is structured as follows: Chapter 2 establishes the foundational RL terminology. Chapter 3 reviews current literature and related works. Chapter 4 details the project architecture and technical implementation. Chapter 5 focuses on the systematic evaluation and comparative analysis of the tested algorithms. Finally, ?? provides the conclusions and outlines for potential future expansions of the framework.

2

Related Work

The domain of autonomous target tracking using mobile service platforms has experienced a significant paradigm shift, transitioning from passive sensor data aggregation to active, information-driven path planning. In tracking environments constrained by sparse or incomplete sensory feedback, such as Range-Only Single-Beacon (ROSB) localization, this active component is critical. Because raw distance measurements lack directional components, a domestic service robot cannot rely on traditional stationary estimation; to figure out exactly where it is using just a single signal beacon, a robot can't just drive in a straight line. Because indoor spaces like rooms and hallways naturally force it to move straight, the robot has to intentionally break out of those patterns, making sharp curves or other deliberate movements in order to strip away the mathematical guesswork and pinpoint its location.

To contextualize the integration of estimation and guidance evaluated in this thesis, this chapter reviews the historical and contemporary state of the art. While the mathematical foundations of single-beacon observability date back to classical marine localization studies, this review is specifically anchored by two modern milestones that successfully transition these principles into active, closed-loop applications: the robust probabilistic estimation framework established in *“Range-Only Single-Beacon Tracking of Underwater Targets From an Autonomous Vehicle: From Theory to Practice”* (Masmitja et al., 2019), and the learning-based guidance architecture developed in *“A reinforcement learning path planning approach for range-only underwater target localization with autonomous vehicles”* (Masmitja et al., 2022).

By synthesizing these anchor works alongside the broader historical landscape, this chapter analyzes the literature across three distinct thematic domains: the evolution of deterministic and probabilistic tracking layers within range-only frameworks; the computational bounds of classical information-theoretic active perception; and the deployment of Reinforcement Learning (RL) for continuous robotic guidance. Ultimately, this chapter establishes the explicit methodological motivation for translating the algorithmic lessons of these marine frameworks into indoor, room-based service robotics.

2.1 Evolution of ROSB Estimation Techniques

The challenge of inferring a target's continuous kinematic trajectory from scalar distance measurements has long been categorized in localization literature as a highly non-linear inversion problem. Historically, the ROSB framework was organically born out of the extreme physical constraints of marine engineering. Because electromagnetic signals like GPS are rapidly attenuated by seawater within millimeters, underwater vehicles operate in an absolute GPS-denied environment. While multi-point triangulation could be achieved via pre-deployed Long Baseline (LBL) acoustic arrays (Vaganay et al., 2004), their immense deployment, calibration, and hardware costs drove the marine community

to seek a minimal infrastructure alternative: a singular acoustic transceiver measuring the scalar Time-of-Flight (range) to a single beacon.

This physical constraint introduces a severe geometric symmetry problem. A single scalar distance reading contains no angular data, restricting the target's possible locations to an omnidirectional hypersurface of spatial uncertainty, famously visualized as a "hollow ring" geometry in 2D space, as shown in Figure 2.1.

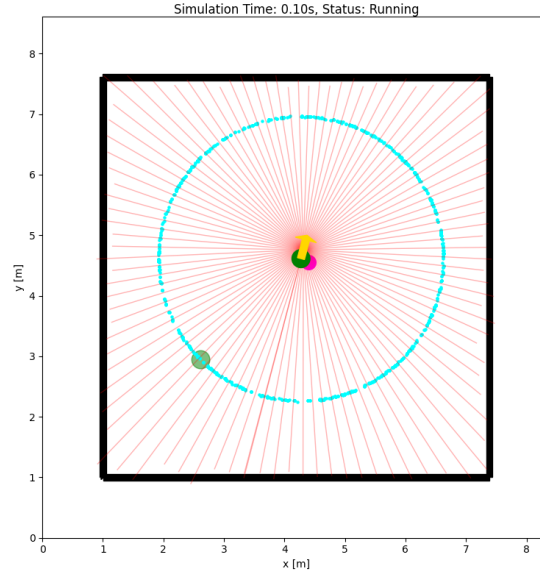


Figure 2.1. The hollow ring of uncertainty in an IR-Sim environment

Early tracking frameworks addressed this non-linear inversion through a geometric lens, relying heavily on batch-windowed optimization architectures. The foundational roots of single-beacon tracking began with Scherbatyuk (1995), who proposed early least-squares schemes to estimate platform positions under unmodeled ocean currents. This evolved into the concept of Synthetic Long Baseline (SLBL) navigation by Larsen (2000), which synthetically recreated multi-beacon triangulation by compounding successive, time-separated range readings from a single moving vehicle. Shortly thereafter, Newman & Leonard (2003) successfully extended these underlying mechanics into the domain of pure range-only Simultaneous Localization and Mapping (SLAM).

Crucially, the theoretical bounds of this tracking paradigm were formalized by Gadre & Stilwell (2004, 2005), who mathematically proved the non-linear observability criteria of single-beacon systems. Their proofs conclusively demonstrated that if an observing vehicle moves along a straight line-of-sight path directly toward or away from a beacon, cross-range unobservability remains absolute. Resolving this challenge requires the platform to execute dynamic, geometry-breaking trajectories to actively collapse the spatial ambiguity of the hollow ring.

When evaluated against these strict observability constraints, classical deterministic point-estimators exhibit severe structural vulnerabilities. Because windowed **Least Squares (LS)** frameworks and moving-horizon Gauss-Newton approximations rely on

localized gradient descent mechanics, they are highly sensitive to initialization values. In sparse sensory contexts where early range data is geometrically symmetric, these point-estimators frequently succumb to local minima. This leads to irreversible filter divergence if the target executes unmodeled maneuvers or breaks line-of-sight, as the deterministic estimator cannot map multiple competing spatial hypotheses.

To decouple tracking stability from initialization accuracy, the literature shifted toward probabilistic sequential Monte Carlo tracking, a transition thoroughly validated in real-world marine environments by Masmitja et al. (2019).

2.2 Classical Active Perception

Because the accuracy of an ROSB estimator is fundamentally coupled to the kinematics of the observing platform, target tracking cannot be treated as a passive data-gathering exercise. If an autonomous service robot moves purely along the line-of-sight vector toward a target, cross-range unobservability remains absolute, rendering the underlying filter incapable of resolving spatial ambiguities. Resolving this challenge requires the guidance system to operate under the paradigm of active perception, planning trajectories explicitly designed to maximize the angular diversity of successive range measurements around environmental obstacles.

Historically, the robotics community has framed this information-seeking trajectory generation as an online optimization problem rooted in classical information theory. This literature broadly bifurcates into two mathematical methodologies. The first relies on maximizing the **Fisher Information Matrix (FIM)** or, equivalently, minimizing its inverse, the Cramér-Rao Bound (CRB). This geometric approach optimizes future control sequences by ensuring the robot maneuvers into spatial configurations where the sensitivity of the scalar range sensor relative to the target’s state vector is maximized. While computationally elegant when paired with traditional linear or Gaussian estimators, the FIM framework relies heavily on localized Jacobian matrices. Consequently, it suffers a severe structural breakdown when forced to operate over the highly non-linear, multi-modal belief distributions required to handle signal anomalies.

To accommodate these non-Gaussian spaces, an alternative school of thought utilizes direct **Information-Theoretic Entropy Reduction**. Rather than relying on localized gradients, these frameworks optimize trajectories by minimizing the Shannon Entropy or maximizing the Kullback-Leibler (KL) divergence computed directly over the estimator’s full probability distribution. While mathematically rigorous and highly capable of representing competing spatial hypotheses, contemporary literature identifies a severe **computational lookahead bottleneck** inherent to this paradigm. Projecting a non-Gaussian belief state, such as a dense particle cloud, through a multi-step forward horizon requires calculating expected information metrics across an exponentially expanding decision tree of future control actions.

In complex, real-time reactive scenarios, this overhead quickly exceeds the processing thresholds of standard embedded robotic platforms. When a domestic service robot must simultaneously compute these dense numerical integrations while executing low-level obstacle avoidance within a cluttered room layout, the online optimization pipeline experiences severe latency. As a result, classical information-driven path planning remains largely restricted to offline generation, idealized free-space simulations, or highly truncated lookahead horizons that fail to account for long-term target maneuvers.

2.3 Learning-Based Active Guidance

To circumvent the computational limitations of online numerical optimization in ambiguous environments, modern autonomous guidance research has increasingly turned toward model-free **Reinforcement Learning (RL)** (Masmitja et al., 2022). Because the target’s true kinematic state remains hidden from the platform, the navigation task is strictly formulated as a Partially Observable Markov Decision Process (POMDP).

Mapping these continuous action spaces to smooth velocity profiles requires robust actor-critic architectures. Maximum-entropy frameworks like **Soft Actor-Critic (SAC)** leverage policy entropy regularization to actively prevent the agent from trapping itself in repetitive, sub-optimal geometric paths. Concurrently, conservative value estimators like **Twin Delayed Deep Deterministic Policy Gradient (TD3)** are deployed to mitigate value function overestimation when the training loop is subjected to the heavily corrupted, acoustic data typical of marine environments.

While the network architecture dictates the learning mechanics, the emergent trajectory behavior is governed entirely by reward engineering. Contemporary tracking frameworks reject raw distance minimization, as a direct approach fails to collapse geometric ambiguities. Instead, modern reward functions are structured as composites that prioritize cross-range acceleration components to induce geometry-breaking trajectories.

2.4 Marine-to-Indoor

The systemic bifurcation between advanced state estimation and real-time autonomous guidance becomes uniquely acute when translating active perception frameworks from open marine environments to bounded indoor settings. In deep-sea operations, range-only tracking algorithms rely on the luxury of large-scale, unconstrained geometric trajectories to resolve spatial ambiguities. In contrast, a domestic service platform must execute these geometry-breaking maneuvers within tight, obstacle-dense room layouts, confronting severe non-line-of-sight occlusions and multi-path signal reflections caused by structural walls while simultaneously maintaining localized obstacle avoidance.

This thesis directly aims to adapt the core structural philosophy of marine ROSB frameworks for indoor service robotics. This work couples a linear and non-linear, multi-modal probabilistic tracking backend directly with a continuous reinforcement learning guidance layer, evaluating how effectively learning-based active perception policies can interpret, leverage, and collapse complex spatial uncertainties under the strict physical and geometric boundaries of domestic indoor layouts.

Theoretical Framework

3.1 Hidden Markov Models

To successfully navigate an environment characterized by sparse or incomplete sensory feedback, an autonomous vehicle cannot merely rely on passive data collection; it must engage in **Active Perception**, deliberately executing control policies that maximize information gain and actively break spatial ambiguities (Bajcsy et al., 2018). This strategy fundamentally depends on robust state-estimation frameworks to translate raw, noisy sensor data into a reliable, dynamically updated spatial representation (Thrun et al., 2005). In target tracking and localization domains, this requires processing time-series observations to recursively infer a system’s true kinematic variables over time (Bar-Shalom et al., 2001).

This section establishes the mathematical and theoretical foundations of the tracking methodologies evaluated throughout this framework to drive these information-seeking behaviors. The estimation problem is first formalized under the structural lens of Hidden Markov Models (HMMs). Following this baseline, two distinct paradigms for resolving the target’s position are explored: a traditional deterministic optimization approach utilizing Least Squares (LS) (Simon, 2006) estimation, and a fully probabilistic, sequential Monte Carlo approach leveraging Particle Filtering (PF) (Arulampalam et al., 2002). Together, these methodologies provide the underlying belief state updates that allow the overarching reinforcement learning framework to plan optimal active perception trajectories.

3.1.1 Target Tracking Formulation

In the context of Range-Only Single-Beacon (ROSB) navigation, the tracking problem is explicitly structured as a discrete-time **Hidden Markov Model (HMM)**. This formulation represents a system where the true state sequence $\{X_1, X_2, \dots, X_t\}$ is unobservable (hidden), and can only be inferred through a corresponding sequence of noisy observations $\{Y_1, Y_2, \dots, Y_t\}$. In this specific deployment, these components are defined as:

- **The Hidden State (X_t):** The true, unobservable kinematics of the target (e.g., its relative 2D coordinates and velocities).
- **The Observation (Y_t):** The raw, scalar acoustic slant-range measurements received by the tracking vehicle’s onboard modem.

The framework relies on two primary probabilistic mechanics to resolve this hidden state over discrete time steps:

$$\text{Transition Model: } \mathbb{P}(X_t | X_{t-1}) \quad (3.1)$$

$$\text{Emission/Measurement Model: } \mathbb{P}(Y_t | X_t) \quad (3.2)$$

The transition model encapsulates the system dynamics, dictating how the target moves from one time step to the next under the Markov assumption (where the future state depends strictly on the immediate present state). The emission model defines the physical sensor physics, mapping how a specific hidden position stochastically yields a specific acoustic range reading.

Because a single range measurement creates a non-Gaussian ring of positional ambiguity around a single beacon, a single observation is insufficient to resolve the hidden state. By applying Bayes' rule recursively, the probability distribution function over the hidden states can be solved over time through sequential filtering methods:

$$\mathbb{P}(X_t | Y_{1:t}) \propto \mathbb{P}(Y_t | X_t) \int \mathbb{P}(X_t | X_{t-1}) \mathbb{P}(X_{t-1} | Y_{1:t-1}) dX_{t-1} \quad (3.3)$$

This recursive estimation pipeline continuously refines the belief state of the target's location, which ultimately serves as the foundational observation space input for the overarching MDP control loop.

3.1.2 Least Squares Estimation

A classical, non-probabilistic approach to solving the ROSB localization challenge is the **Least Squares (LS)** method, as seen in Figure 3.1. Unlike recursive filters that update a continuous probability distribution, LS approaches the tracking problem from a geometric and optimization perspective, treating it as a batch or moving-window optimization task.

Given a history of N known beacon deployment locations (or historical tracking vehicle coordinates) $\mathbf{x}_{b,i} = [x_{b,i}, y_{b,i}]^T$ and their corresponding range measurements y_i for $i = 1, \dots, N$, the true position of the target $\mathbf{x} = [x, y]^T$ can be estimated by minimizing the sum of the squared residuals. The non-linear measurement residual function is formalized as:

$$r_i(\mathbf{x}) = y_i - \sqrt{(x - x_{b,i})^2 + (y - y_{b,i})^2} \quad (3.4)$$

The optimal target position estimate $\hat{\mathbf{x}}_{LS}$ is subsequently found by minimizing the objective function $S(\mathbf{x})$:

$$\hat{\mathbf{x}}_{LS} = \arg \min_{\mathbf{x}} S(\mathbf{x}) = \arg \min_{\mathbf{x}} \sum_{i=1}^N [r_i(\mathbf{x})]^2 \quad (3.5)$$

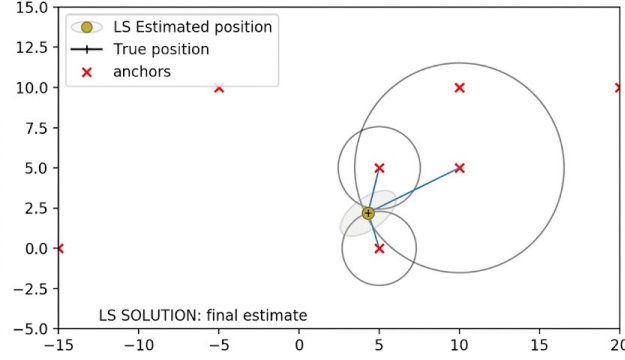


Figure 3.1. Diagram showing the functioning of a Least Squares algorithm.

Because the range model relies on non-linear Euclidean distance formulations, a closed-form analytical solution cannot be achieved directly. Instead, the system must be linearized using Taylor series expansions around a nominal trajectory estimate via iterative numerical optimization, such as the Gauss-Newton or Levenberg-Marquardt algorithms. While computationally direct, LS methods suffer from significant limitations in ROSB setups: they lack a dynamic process model to track targets changing velocities over time and risk severe divergence if the initial optimization guess lands in local minima caused by range ambiguities.

3.1.3 Particle Filtering

To overcome the non-linear, non-Gaussian constraints inherent to range-only single-beacon configurations, the **Particle Filter (PF)**, fundamentally rooted in Sequential Monte Carlo (SMC) methods, provides a robust, fully probabilistic solution to the underlying HMM equations. A diagram of a Particle Filter execution is shown in 3.2. Instead of approximating state estimates as a single linearized point or a rigid Gaussian distribution, the PF models the complex, multi-modal posterior probability distribution $\mathbb{P}(X_t | Y_{1:t})$ using a cloud of M discrete, weighted samples called particles:

$$\mathbb{P}(X_t | Y_{1:t}) \approx \sum_{i=1}^M w_t^{(i)} \delta(X_t - X_t^{(i)}) \quad (3.6)$$

where $X_t^{(i)}$ denotes the kinematic state hypothesis of the i -th particle, $w_t^{(i)}$ represents its corresponding normalized importance weight satisfying $\sum w_t^{(i)} = 1$, and $\delta(\cdot)$ is the Dirac delta function.

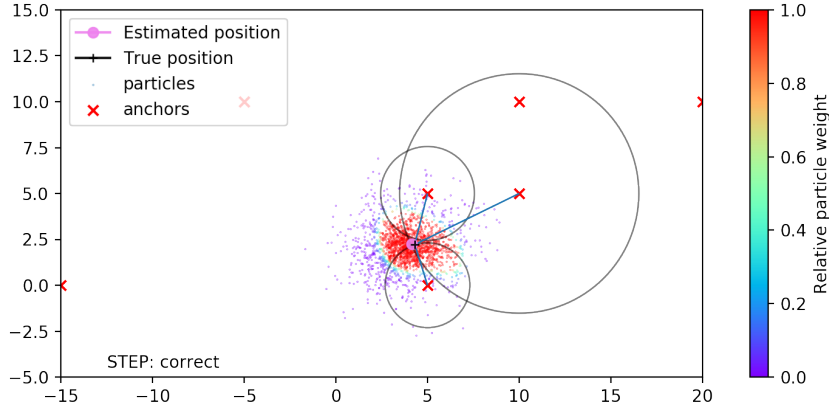


Figure 3.2. Diagram showing the functioning of a Particle Filter algorithm. Credits: (Mikoff, 2020)

The implementation of the Particle Filter tracking pipeline operates recursively over three structural steps at each time step t :

1. **Prediction (Sampling):** Each particle is propagated forward in time through the state transition model, simulating independent kinematic trajectories corrupted by stochastic process noise $\eta_t^{(i)}$:

$$X_t^{(i)} \sim \mathbb{P} \left(X_t \mid X_{t-1}^{(i)} \right) = f \left(X_{t-1}^{(i)} \right) + \eta_t^{(i)} \quad (3.7)$$

2. **Correction (Importance Weighting):** Upon receiving a new acoustic range observation y_t , the importance weight of each particle is updated. This update evaluates how likely the simulated particle position is to produce the actual recorded range measurement, utilizing the sensor's Gaussian measurement noise profile $v_t \sim \mathcal{N}(0, \sigma^2)$:

$$w_t^{(i)} \propto w_{t-1}^{(i)} \cdot \mathbb{P} \left(Y_t = y_t \mid X_t^{(i)} \right) = w_{t-1}^{(i)} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{\left(y_t - h(X_t^{(i)}, \mathbf{x}_{b,t}) \right)^2}{2\sigma^2} \right) \quad (3.8)$$

where $h(\cdot)$ is the non-linear Euclidean distance measurement function.

3. **Resampling:** Over successive iterations, the variance of the importance weights inevitably increases, leading to a phenomenon known as particle degeneracy, where a small subset of particles accumulates the majority of the total weight while others become negligible. To counteract this, a selection step is triggered based on the effective particle size $N_{eff} = \left(\sum (w_t^{(i)})^2 \right)^{-1}$. If N_{eff} drops below a defined threshold (typically $M/2$), low-weighted hypotheses are systematically purged while highly viable particles are duplicated.

The numerical execution of this selection step can rely on several algorithmic resampling schemes, which primarily vary in computational complexity and sampling variance (Li et al., 2015):

- *Multinomial Resampling*: New particles are drawn independently with replacement from the discrete distribution defined by $w_t^{(i)}$. Mapping M uniformly distributed random numbers onto the cumulative sum of weights incurs an $\mathcal{O}(M \log M)$ complexity and introduces a high sampling variance.
- *Residual Resampling*: A two-phase approach where $\lfloor M \cdot w_t^{(i)} \rfloor$ exact copies of particle i are deterministically retained, while the remaining fractional slots are filled stochastically using a secondary sampling scheme over the residual components.
- *Stratified Resampling*: Divides the cumulative weight interval $(0, 1]$ into M disjoint sub-intervals of length $1/M$. Generating independent random variables within each stratum guarantees an execution complexity of $\mathcal{O}(M)$ while strictly minimizing sampling variance.
- *Systematic Resampling*: This scheme is chosen as the operational resampling mechanism for this project due to its ideal optimization for real-time robotics. **Consequently, this scheme is selected as the operational resampling mechanism for the implementation of this project.** The algorithm samples a single random starting point $U_1 \sim \mathcal{U}(0, 1/M]$ and deterministically projects the remaining points at rigid intervals where $U_k = U_1 + \frac{k-1}{M}$. It operates strictly in $\mathcal{O}(M)$ execution time and minimizes tracking latency while ensuring excellent particle diversity.

In ROSB frameworks, the Particle Filter shows exceptional capability. When the tracking vehicle receives its initial range readings, the particle cloud naturally clusters into a physical, hollow ring surrounding the beacon, successfully mirroring the exact geometric ambiguity of the system. As the vehicle navigates and gathers multi-angle range data, the mismatched particle trajectories are structurally pruned away during the correction phase, forcing the particle distribution to collapse onto the true target coordinates. While offering superior tracking accuracy under highly non-linear conditions, the PF introduces a higher computational overhead compared to deterministic LS methods.

3.2 Reinforcement Learning

RL (Sutton & Barto, 2018; Murel & Kavlakoglu, n.d.; Murphy, 2025) is a branch of machine learning and a paradigm of sequential decision making centered on active environmental interaction. Yet, within the broader machine learning landscape, RL stands distinct from both supervised learning, which relies on static, pre-labeled datasets, and unsupervised learning, which focuses on identifying latent structural patterns. The mechanism it employs closely mirrors natural behavioral conditioning, famously illustrated by an animal adapting its actions through trial and error to maximize positive

feedback, such as a reward from an instructor, while learning to avoid penalties. When translated to a computational architecture, an autonomous entity explores an unmapped or uncertain system to discover optimal operational strategies purely through experiential feedback, maximizing a cumulative performance metric over time. This interactive, feedback driven nature makes RL uniquely suited for mobile robotics applications where predetermined paths or perfect environmental models are inherently unavailable.

3.2.1 Foundations and The MDP Framework

The foundational Reinforcement Learning architecture is conceptually structured around a continuous feedback loop between two primary entities: the **Agent** (the autonomous decision-maker or controller) and the **Environment** (everything external to the agent, representing the physical world or system being controlled).

Mathematically, this interaction is formalized as a **Markov Decision Process (MDP)**. An MDP is strictly defined by the 5-tuple (S, A, P, R, γ) , where at each discrete time step t :

- S is the finite or continuous **State Space**, representing all valid configurations of the environment. The specific state at time step t is denoted as $s_t \in S$.
- A is the **Action Space**, defining the set of all possible moves or commands available to the agent. The action executed at time step t is denoted as $a_t \in A$.
- P is the **Transition Probability Function**, defined as $P(s_{t+1} \mid s_t, a_t) = \mathbb{P}(S_{t+1} = s_{t+1} \mid S_t = s_t, A_t = a_t)$. This defines the stochastic probability that the environment will transition to state s_{t+1} given the current state s_t and action a_t . This satisfies the Markov property, meaning the next state depends solely on the immediate current state and action, independent of historical trajectories.
- R is the **Reward Function**, mapping a state-action transition to a scalar value, expressed as $R(s_t, a_t) \in \mathbb{R}$. It provides immediate localized feedback on the utility of action a_t in state s_t . In this work, this function establishes the baseline objective by penalizing undesirable behaviors, such as obstacle collisions derived from HouseExpo layouts, while incentivizing active target compliance.
- γ is the **Discount Factor**, where $\gamma \in [0, 1]$. It determines the current worth of future rewards, ensuring the mathematical convergence of infinite-horizon tasks and modulating the agent's horizon of foresight.

3.2.2 The Algorithmic Interaction Loop

The practical runtime execution of this mathematical structure operates as an iterative feedback pipeline over time steps $t = 0, 1, 2, \dots$, as illustrated in Figure 3.3:

1. **Observation:** The agent perceives the current environmental state $s_t \in S$.
2. **Action Selection:** The agent selects an action $a_t \in A$ according to its operational strategy.

3. **Transition and Feedback:** The environment processes a_t , transitioning stochastically to a new state s_{t+1} via P and emitting a scalar reward r_{t+1} via R .
4. **Optimization:** The agent utilizes the experience trajectory to update its internal parameters.

Throughout this sequential process, the agent must continually manage the **exploration-exploitation trade-off**. It must balance utilizing its current knowledge to select actions known to yield high rewards (exploitation) with attempting novel, unverified actions to discover more optimal, long-term navigation trajectories (exploration), allowing the system to escape local traps and converge toward a global optimum.

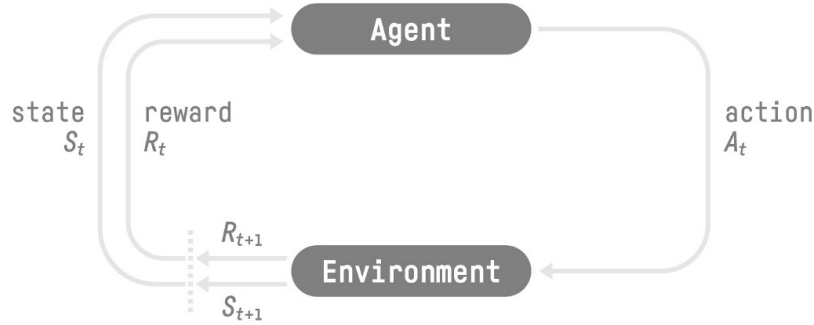


Figure 3.3. Diagram of the sequential Reinforcement Learning interaction loop. Credits: (Simonini & The Hugging Face Team, 2022)

3.2.3 Policies and Value Functions

To make intelligent choices within this loop, the agent relies on an internal policy and uses mathematical value functions to evaluate its decisions over an extended timeline.

Policy

The policy, denoted as π , defines the agent's behavior by mapping a perceived state to a corresponding action selection. A policy can be deterministic, directly mapping a state to a single action ($\pi(s) = a$), or stochastic, defining a probability distribution over the action space given a state ($\pi(a | s) = \mathbb{P}(A_t = a | S_t = s)$). In deep reinforcement learning frameworks, policies are modeled as high-dimensional neural network function approximators that map continuous sensory inputs directly to operational commands.

Discounted Return

To formalize the agent's long-term optimization objective, the sequence of future rewards emitted by the loop is aggregated into a single scalar metric known as the **Discounted Return**, denoted as G_t :

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (3.9)$$

State-Value and Action-Value (Q) Functions

To evaluate the efficacy of its policy, the agent estimates its expected long-term performance using two distinct formulations:

- **State-Value Function ($V^\pi(s)$):** Represents the expected total discounted return an agent will accumulate starting from a specific state s and strictly following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi [G_t | s_t = s] \quad (3.10)$$

- **Action-Value Function ($Q^\pi(s, a)$):** Quantifies the expected discounted return achieved by taking an arbitrary action a in state s , and subsequently adhering to policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | s_t = s, a_t = a] \quad (3.11)$$

The Q -function is of paramount importance to the architectures evaluated in this thesis (such as TD3, SAC, and DDPG). These operate on an Actor-Critic paradigm, where a neural network "Critic" specifically approximates $Q(s, a)$ to guide and optimize the policy changes of the "Actor".

The Bellman Expectation Equation

Both $V^\pi(s)$ and $Q^\pi(s, a)$ satisfy a fundamental recursive relationship known as the **Bellman Expectation Equation**. This formulation decomposes the value of a current state-action pair into the immediate reward plus the discounted expected value of the succeeding state:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) \sum_{a' \in A} \pi(a' | s') Q^\pi(s', a') \quad (3.12)$$

This recursive decomposition enables the agent to iteratively update its long-term estimates at every discrete interaction step, serving as the mathematical bedrock for all Temporal Difference learning algorithms.

3.2.4 Taxonomy of Reinforcement Learning Approaches

Before exploring specific algorithmic implementations, implementations are fundamentally bifurcated by whether they explicitly use the environmental transition function P . Frameworks that construct or utilize internal transition approximations are classified as *model-based* and leverage them for predictive planning. Conversely, *model-free* architectures learn strictly through experiential trial-and-error. Within the model-free class, algorithms are categorized by the underlying mechanism they optimize.

Value-Based Methods

Value-based approaches focus entirely on approximating the optimal action-value function ($Q^*(s, a)$), deriving an implicit policy by selecting actions that maximize those expected returns. These methods rely heavily on the fundamental **SARS** experience tuple, denoted as $(s_t, a_t, r_{t+1}, s_{t+1})$.

A foundational example is **SARSA** (State-Action-Reward-State-Action), an on-policy temporal difference method updating estimates using the actual next action executed. Its off-policy counterpart, Q-learning, updates estimates assuming absolute greedy choices. While highly efficient for discrete tasks, vanilla value-based methods struggle to scale to the continuous action spaces (such as linear and angular robotic velocities) required in modern navigation.

Policy-Based Methods

Instead of maintaining a value function, policy-based methods directly optimize the parameters θ of a policy network $\pi_\theta(a | s)$ using gradient ascent to maximize a performance scalar $J(\theta)$, formulated via the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q^{\pi_\theta}(s, a)] \quad (3.13)$$

Directly parameterizing the policy allows these methods to naturally handle continuous action spaces. However, they frequently suffer from high variance in gradient estimates and poor sample efficiency, as trajectories cannot easily be reused once the policy parameters shift.

Actor-Critic Methods

To alleviate the high variance of policy-based methods and the scaling limitations of value-based methods, the hybrid **Actor-Critic** paradigm combines both approaches, splitting the workload between two distinct neural networks:

- **The Actor:** Represents the parameterized policy $\pi_\theta(s)$, responsible for observing states and selecting continuous actions.
- **The Critic:** Represents a value function approximator ($Q_\phi(s, a)$), responsible for evaluating the actions selected by the actor.

The continuous control algorithms evaluated in this work, Twin Delayed DDPG (**TD3**), Soft Actor-Critic (**SAC**), Deep Deterministic Policy Gradient (**DDPG**), and Proximal Policy Optimization (**PPO**), are fundamentally rooted in this hybrid framework.

Continuous Control Algorithms: From DDPG to TD3 and SAC

To address the continuous action spaces inherent to robotic navigation, this thesis evaluates two primary paradigms of deep Actor-Critic architectures: deterministic policies and maximum-entropy stochastic policies.

Deep Deterministic Policy Gradient (DDPG) & Twin Delayed DDPG (TD3)

DDPG adapts the principles of deterministic policy gradients to deep neural networks. Unlike stochastic policies, the actor directly maps a state to a single continuous action command: $a = \mu_\theta(s)$. While highly efficient, DDPG notorious suffers from severe overestimation bias in its value function estimates, leading to policy divergence in complex environments.

To mitigate these instabilities, **TD3** introduces three structural optimizations:

- **Clipped Double Q-Learning:** The architecture maintains two independent critics, Q_{ϕ_1} and Q_{ϕ_2} , and utilizes the minimum of their estimates to calculate the temporal difference target, penalizing overestimation.
- **Delayed Policy Updates:** The actor networks and target networks are updated less frequently than the critic networks, ensuring the value estimates stabilize before the policy changes direction.
- **Target Policy Smoothing:** A small amount of random noise is added to the target action calculation, smoothing the value function landscape and preventing the policy from exploiting sharp localized peaks.

Soft Actor-Critic (SAC) In contrast to the deterministic nature of TD3, **SAC** is an off-policy, stochastic framework based on the Maximum Entropy RL paradigm. Instead of merely maximizing the expected long-term return, SAC augments the objective function with an information-theoretic entropy regularizer $\mathcal{H}$:

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))] \quad (3.14)$$

where α is the temperature parameter governing the trade-off between reward optimization and exploratory entropy. By explicitly incentivizing the policy to remain as random as possible while capturing high rewards, SAC prevents premature convergence to local minima, balances robust exploration, and exhibits superior resilience when dealing with the non-Gaussian observation spaces produced by ROSB filtering.

4

Methodology

4.1 Development and Architecture

This section details the development of the **ROSB Localization Project** (Tommasi, 2026), tracing the architectural pipeline from raw structural floorplans to a fully interactive, learning-based navigation environment. The workflow initiates with the offline translation of 2D layouts from the benchmark **HouseExpo** (Li et al., 2020) dataset into the native YAML configuration format utilized by the **IR-Sim** (Han et al., 2026) simulation engine to generate compatible collision maps. To permit standardized **RL** (Sutton & Barto, 2018) loops, this simulator is encapsulated within a custom **Gymnasium** (Towers et al., 2024) environment. This wrapper formally defines the system’s observation space, action space, reward structures, and step transitions, while utilizing **Stable Baselines3** (Raffin et al., 2021) to implement the continuous control agents (DDPG, PPO, TD3, SAC).

Operating within this wrapper, the online architecture tightly couples state estimation with autonomous guidance (Bajcsy et al., 2018). The perception layer features two distinct estimation algorithms: a Linear **Least Squares** (LS) (Simon, 2006) filter and a probabilistic **Particle Filter** (PF) (Arulampalam et al., 2002), which process noisy, range-only measurements to infer hidden target kinematics. The resulting belief states and filter covariances directly feed the reinforcement learning policy network, which maps these observations to optimal geometric waypoints using a composite reward framework designed to maximize target tracking efficiency while minimizing wall collisions. To bridge the gap between these high-level policy commands and continuous motor execution, an integrated **A*** (Foead et al., 2021) pathfinding algorithm evaluates the inflated collision map to compute collision-free trajectories.

The progression of this section mirrors this development flow. The foundational dataset parsing and the programmatic structure of the environment wrapper are established first, setting the stage for the mathematical formalization of the LS and PF tracking algorithms. With the estimation backend defined, the focus shifts to the reinforcement learning architecture, where the observation spaces, actor-critic networks, and information-gain reward structures are broken down. Finally, we show how the A^* local planner safely bridges the estimated target goal position with the current state of the robot, enabling more efficient real-time navigation.

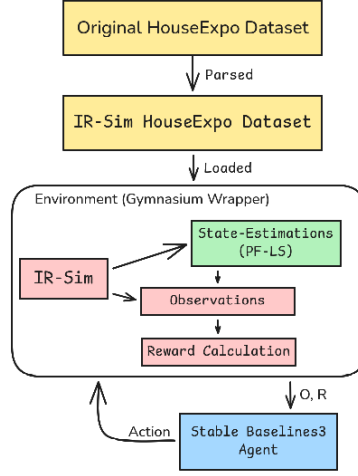
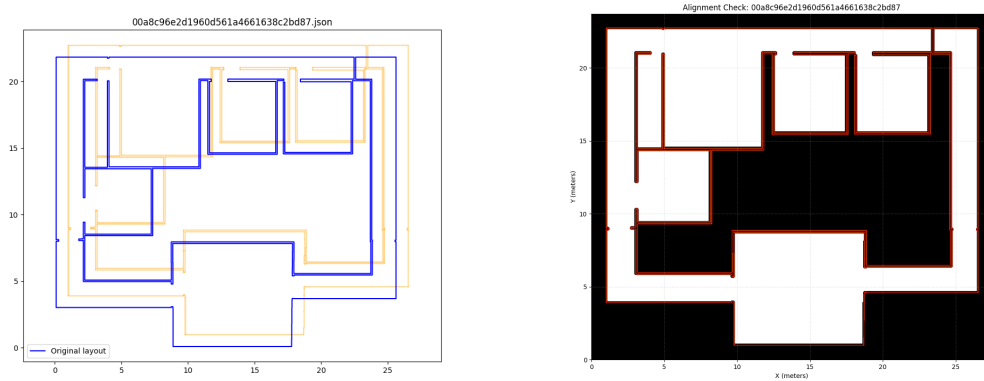


Figure 4.1. System architecture mapping the offline data pipeline and the environment interaction loop.

4.1.1 Map Parsing and Occupancy Grid Generation

As outlined in the architectural overview, the initial phase requires translating raw spatial data into both an interactive simulation format and a discrete mapping layer. To evaluate the active perception framework across varied, realistic architectural layouts, an automated data pipeline was developed to port the benchmark HouseExpo dataset into both the native YAML configuration utilized by the IR-Sim engine and corresponding 2D occupancy grids. The final result of this parsing procedure can be viewed in Figure 4.2a and Figure 4.2b.



(a) Original (blue) and the converted (yellow) maps. **(b)** Generated occupancy grid with inaccessible (black) and accessible (white) areas.

Figure 4.2. Example of the map conversion process.

Raw floor plans within the HouseExpo dataset are originally stored as JSON files containing sequences of polygonal bounding vertices. The conversion pipeline programmatically parses these vertices and applies a rigid geometric transformation to establish a standardized simulation environment. First, a global bounding box is computed for each layout to determine absolute spatial limits. The entire vertex set is then shifted into a positive coordinate reference frame, and the world boundaries are padded with a 1.0 m peripheral safety margin to prevent boundary-collision artifacts during edge-case maneuvers.

To instantiate physical boundaries within the continuous simulation, the 1D line segments connecting consecutive vertices are transformed into 2D static rectangular obstacles. For each consecutive vertex pair, the Euclidean distance defines the wall’s length, while a uniform structural thickness of 0.1 m is applied. The physical center coordinates (x, y) and rotational orientation (θ) of each rectangular wall are mathematically derived using the segment’s midpoint and arctangent. Alongside the structural layout, the pipeline injects the kinematic and sensory configurations for the service platform directly into the YAML files. The agent is defined as a differential drive platform with a simulated 2D LiDAR array casting 100 uniformly spaced beams up to a 7.0 m maximum range, with synthetic Gaussian noise ($\sigma = 0.05$) explicitly applied to realistically replicate real environmental noise.

To ensure the statistical validity of the trained agents, the processed dataset is then partitioned into three distinct subsets: Training, Evaluation, and Testing. To maintain reproducibility across all experimental phases, the file list is deterministically sorted before applying a fixed random seed to the shuffling algorithm. Out of the total 35,126 layouts, the pipeline systematically isolates 500 maps for final policy evaluation and 3,000 maps for blind testing, leaving the remaining 31,626 maps as a high-variance training distribution. This partition strategy ensures that the reinforcement learning policy is exposed to a sufficiently broad architectural variety while maintaining dedicated, unseen environments for rigorous quantitative benchmarking.

In parallel to the YAML generation, the pipeline renders discrete Occupancy Grids required for the A^* pathfinding routing, Robot/Goal initial pose generation and PF particles out of bounds checks. The continuous world coordinates are discretized into a high-resolution grid matrix at a resolution of 0.05 m per pixel using OpenCV rasterization algorithms. The internal floor space is identified by applying an XOR parity rule over the filled polygons, followed by a bounding flood-fill algorithm originating from the padded exterior margin to define structurally enclosed rooms. Crucially, to guarantee physical safety during real-time tracking maneuvers, a Configuration Space (C-Space) filtering algorithm is applied to the rasterized grid. Using morphological image processing, the physical obstacles are dilated by an elliptical kernel sized to the structural radius of the robot plus an additional safety clearance margin (0.35 m total). This geometric operation artificially inflates walls and safely seals narrow corridors or doorways that the physical platform cannot traverse. A connected components analysis is then executed over the remaining free space to isolate the largest contiguous navigable room, discarding isolated unreachable pockets.

The final processed layouts are exported as binary NumPy arrays and PNG image masks, accompanied by metadata files to perfectly align the discrete grid coordinate frame with the continuous IR-Sim environment.

4.1.2 Gymnasium Environment Encapsulation

To integrate the customized simulation framework with modern Reinforcement Learning (RL) algorithms, the engine was encapsulated within a custom Python class implementing the `gymnasium.Env` interface.

At the beginning of each episode, the `reset()` function initializes a new scenario by randomly selecting one of the translated HouseExpo environments. Both the robot and the hidden target are assigned valid starting positions, dynamically verified against the inflated C-Space occupancy grid to prevent geometric collisions. Constraints such as minimum separation distance and the forced absence of direct line-of-sight are enforced to ensure challenging initial localization conditions.

The environment follows a fixed, discrete-time execution cycle. During each `step()` call, the wrapper synchronizes several subsystems sequentially: it applies the policy’s velocity commands to the simulator, updates the kinematic state of the moving target, acquires the latest LiDAR scan, and feeds the new range-only measurement into the selected target localization algorithm. Furthermore, the environment internally manages a hierarchical navigation state machine. Once the target estimate satisfies predefined confidence criteria, the environment invokes an integrated A* planner to compute a collision-free path over the occupancy grid, extracting simplified intermediate waypoints for the agent to follow.

Finally, the `step()` function compiles the current state of these subsystems into a fixed-dimensional observation vector, calculates the scalar reward signal, and evaluates terminal conditions (e.g., successful target docking, obstacle collision, or maximum step timeout). Alongside these standard Markov Decision Process (MDP) outputs, the environment logs diagnostic metrics such as traveled distance and localization error for statistical evaluation.

4.1.3 Estimation Backend

Operating entirely within the Gymnasium wrapper, the state estimation layer is structured as a modular architecture designed to handle the severe observability constraints of Range-Only Single-Beacon (ROSB) tracking. The framework exposes a configuration parameter that allows the user to dynamically select the active tracking algorithm prior to initializing the environment.

The foundational algorithms for this backend were imported directly from an external script developed for a previous localization project (Masmitja, 2021). Depending on the experimental requirements, the user can configure the wrapper to utilize one of two distinct methodologies for the duration of the run:

- **Linear Least Squares (LS) Filter:** A deterministic point-estimator that solves localized optimization problems over a sliding history window. This option provides a high-speed, computationally lightweight target estimate. Implemented with a window of just 3 samples.
- **Sequential Monte Carlo Particle Filter (PF):** A probabilistic filter that maintains a dense cloud of sample particles to explicitly resolve the multi-modal spatial ambiguities inherent to range-only tracking.

While the core mathematical logic was imported, critical modifications were applied to the original Particle Filter implementation to ensure robust integration with the newly developed simulation environment and discrete occupancy grids. Specifically, the particle initialization and resampling routines were structurally hardened:

- **Spatial Bounding and Obstacle Avoidance:** A validation mask was integrated into the particle generation step. This modification continuously cross-references the environment’s C-Space occupancy grid, strictly preventing particles from spawning out of bounds or inside non-traversable physical walls. This ensures the filter’s probability distribution is exclusively restricted to valid navigable space.
- **Divergence Fallbacks:** Safety mechanisms and exception handling were implemented to prevent runtime execution errors in edge-case scenarios where no valid particles can be spawned (e.g., during severe filter divergence or when the probability space temporarily collapses due to extreme sensor noise).

These programmatic hardening steps were absolutely necessary to guarantee the uninterrupted stability of the Reinforcement Learning framework, preventing fatal crashes during training sessions that span millions of continuous simulation steps. Also they allowed the PF to converge in a much quicker manner.

4.1.4 Reinforcement Learning Framework

The navigation problem is formally defined as a Partially Observable Markov Decision Process (POMDP), where the robot must reach a hidden target whose position is only indirectly inferred. Because the true target location is not directly observable, standard MDP-based learning algorithms cannot be applied natively. To resolve this, the architecture converts the POMDP into a continuous Belief-MDP: the reinforcement learning policy operates entirely on a dynamic belief state generated by the estimation backend described in Section 4.1.3, bridging the gap between the partially observable reality and the fully observable inputs required by Stable Baselines3.

To investigate the suitability of different continuous control methods for this task, four reinforcement learning algorithms available in the Stable Baselines3 framework were evaluated: **Soft Actor-Critic (SAC)**, **Twin Delayed Deep Deterministic Policy Gradient (TD3)**, **Deep Deterministic Policy Gradient (DDPG)**, and **Proximal Policy Optimization (PPO)**.

These algorithms were selected because they represent some of the most widely adopted model-free approaches for continuous robot control, exhibiting diverse exploration strategies and sample efficiencies.

By leveraging the hierarchical state machine managed internally by the Gymnasium wrapper, the RL agent is relieved of global pathfinding duties. The policy is responsible exclusively for local motion control, interpreting sensor data to follow the active waypoint, avoid immediate obstacles, and safely execute the final approach. This decomposition substantially reduces the complexity of the learning problem while retaining the adaptability of learned navigation behaviors.

A diagram of the RL Framework is shown in Figure 4.3

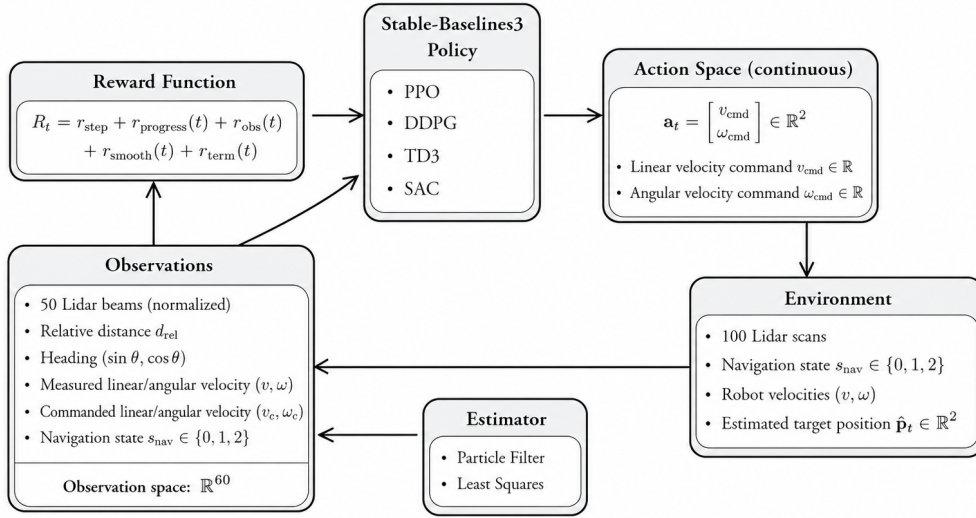


Figure 4.3. The Reinforcement Learning architecture.

Observation Space

At every simulation step, the policy receives a fixed-dimensional observation vector combining perception, navigation, and robot state information. Instead of directly processing the complete localization output (such as the full particle distribution), the observation consists of compact features that provide sufficient information for decision-making while maintaining a low computational overhead.

The observation vector includes:

- A downsampled representation of the 360° LiDAR scan, describing the local obstacle configuration around the robot.
- The relative distance and orientation of the current navigation goal, encoded through the distance together with the sine and cosine of the heading error.

- The commanded linear and angular velocities together with the measured robot velocities.
- The current navigation state, indicating whether the robot is searching for a reliable target estimate, following a planned path, or performing the final docking maneuver.

This representation provides the policy with both low-level sensory information and higher-level navigation context without explicitly exposing the internal mechanics of the localization algorithm.

Action Space

The action space is continuous and consists of two control variables corresponding to the robot's linear and angular velocities. At each decision step, the policy outputs a pair of normalized commands that are directly executed by the simulator for the duration of the control interval.

This formulation allows the reinforcement learning agent to learn smooth motion behaviors while remaining independent of the global planner. High-level navigation objectives, such as waypoint selection and path generation, are handled entirely by the environment's internal logic, whereas the learned policy focuses exclusively on local trajectory execution and obstacle avoidance.

Reward Function

A dense reward function was designed to encourage efficient and safe navigation while ensuring stable policy convergence. Rather than relying solely on sparse terminal rewards, intermediate feedback is provided throughout the episode. Mathematically, the reward at time step t is formulated as the sum of five distinct components:

$$R_t = r_{step} + r_{prog}(t) + r_{obs}(t) + r_{smooth}(t) + R_{term}(t) \quad (4.1)$$

where r_{step} is a constant negative penalty applied at each step to encourage time efficiency. The remaining terms are dynamically computed based on the state of the robot.

State-Aware Progress Reward (r_{prog}) To ensure smooth transitions between global navigation and local target tracking, the distance and heading rewards depend on the active phase of the hierarchical state machine:

$$r_{prog}(t) = \begin{cases} w_p (||e_{t-1}|| - ||e_t||) + w_h (|\theta_{t-1}| - |\theta_t|) & \text{if state is DOCKING} \\ w_p (d_{t-1} - d_t) & \text{otherwise} \end{cases} \quad (4.2)$$

During standard waypoint following, progress is measured purely by the reduction in distance d to the active waypoint.

However, during the final *DOCKING* phase, the objective shifts. The agent must maintain a specific standoff distance $d_{standoff}$ rather than reaching 0.0. Thus, $\|e_t\| = |d_t - d_{standoff}|$ represents the standoff error, and θ_t represents the heading error relative to the target. w_p and w_h are the respective weighting coefficients.

Stable Obstacle Avoidance (r_{obs}) To prevent collisions while allowing narrow maneuvers, the 360° LiDAR scan is partitioned into a forward-facing cone and lateral side cones. Penalties are applied only when obstacles breach predefined safety margins:

$$r_{obs}(t) = -w_{front} \max(0, d_{safe_f} - d_{front}) - w_{side} \max(0, d_{safe_s} - d_{side}) \quad (4.3)$$

where d_{front} and d_{side} are the minimum recorded distances in their respective regions, and d_{safe} dictates the penalty threshold.

Kinematic Smoothness (r_{smooth}) To discourage erratic, oscillatory behaviors ("wiggling") common in continuous RL, a penalty is applied to both the absolute angular velocity ω_t and the discrete derivative of the action vector $a_t = [v_t, \omega_t]^T$:

$$r_{smooth}(t) = -w_{rot}|\omega_t| - w_{wiggle} \sum |a_t - a_{t-1}| \quad (4.4)$$

Milestone and Terminal Logic (R_{term}) Finally, sparse rewards and penalties are triggered upon specific state transitions or episode terminations:

$$R_{term}(t) = \begin{cases} R_{goal} & \text{if } \|e_t\| \leq \epsilon_{dist} \text{ and } |\theta_t| \leq \epsilon_{heading} \\ R_{wp} & \text{if } d_t < 0.5 \text{ (waypoint reached)} \\ R_{coll} & \text{if physical wall collision occurs} \\ R_{breach} & \text{if } d_t < (d_{standoff} - \epsilon_{dist}) \text{ (target rammed)} \\ 0 & \text{otherwise} \end{cases} \quad (4.5)$$

This stage-dependent formulation effectively maps the complex, multi-objective navigation task into a continuous gradient that the Actor-Critic networks can reliably optimize.

4.1.5 Path Planning and Waypoint Pruning

To bridge the gap between high-level localization belief states and local motion execution, an integrated pathfinding framework evaluates the spatial occupancy grid to compute collision-free global trajectories. When the estimation backend satisfies the state machine's confidence criteria, the environment invokes a modified A^* grid-search routine combined with a geometric pruning pipeline. This architecture ensures that the agent is provided with clean, sparse guidance waypoints that optimize runtime computational efficiency while maintaining high safety margins.

Search and Soft Inflation

The pathfinding sequence translates continuous world coordinates into discrete matrix coordinates using the grid resolution scale factor Δ_{res} . Before initiating queue expansion, an early-failure validation routine checks the initial coordinate and target goal coordinate against the raw occupancy grid matrix. If either position intersects a structural boundary, the function terminates early to preserve processor cycles.

The algorithm explores a standard 8-way connectivity graph. To ensure kinematic realism, actions are weighted using asymmetric baseline step costs: orthogonal movements incur a unitary step cost ($c_{\text{orth}} = 1.0$), whereas diagonal movements are penalized by an irrational scaling factor ($c_{\text{diag}} \approx 1.414$) to account for true Euclidean displacements. To guarantee that the calculated trajectories remain robust against modeling errors or sudden tracking drift, the state graph incorporates a two-tiered safety framework:

- **Hard Boundary Constraints:** The search space is strictly bounded by the raw occupancy grid mask. If an expanded neighbor node is classified as out of bounds or situated inside a physical wall, it is discarded immediately.
- **Soft Cost Inflation Buffers:** Rather than treating the Configuration Space (C-Space) inflation zone as an impenetrable wall that restricts path options in tight spaces, nodes falling within the dilated inflation buffer are heavily penalized by an additive scalar cost parameter β_{wall} . This soft cost function shifts the optimal path selection toward the geometric center of corridors and open rooms, actively discouraging the agent from clipping corners or hugging physical boundaries, without completely blocking traversable narrow channels.

To expedite graph convergence and break ties within wide, uniform open spaces, the cumulative node valuation function $f(n)$ utilizes an analytical distance heuristic slightly over-weighted by an inflationary multiplier ϵ_{tie} :

$$f(n) = g(n) + \epsilon_{\text{tie}} \cdot \mathcal{H}(n, g_{\text{grid}}) \quad (4.6)$$

where $g(n)$ is the accumulated path cost from the starting node to current node n , and $\mathcal{H}$ represents the standard Euclidean hypotenuse distance to the discrete goal node g_{grid} . Finally, to insulate the main execution thread from infinite loops caused by dynamically trapped targets or unresolvable labyrinth geometries, an upper execution ceiling regulates the maximum permissible node expansions before forcing a failure fallback.

Compression and LOS Pruning

The raw output of an 8-directional A^* search is inherently jagged, dense, and spoils the reinforcement learning policy to just follow the path without learning the rules. To resolve this, a two-stage compression pipeline filters the dense coordinate sequence into a sparse set of clean guidance waypoints.

The first stage implements a **Line-of-Sight (LOS) Pruning** technique designed to eliminate unnecessary intermediate nodes along clear trajectories.

Beginning at the origin node, the algorithm projects look-ahead vectors to subsequent down-stream path nodes. A linear interpolation raycasting function samples points along the projected segment at half-resolution steps, checking each sample position against the valid navigable space configuration. If a clear line of sight is verified between the current waypoint and a distant look-ahead node, all intervening nodes are safely deleted. The look-ahead pointer advances iteratively, locking in a new waypoint only when a physical boundary intersects the linear projection.

The second stage utilizes a **Spatial Centroid Clustering** routine to prevent redundant waypoint accumulation in confined spaces or around sharp corners. Using a minimum Euclidean distance parameter $d_{\min}$, the algorithm evaluates the remaining nodes sequentially. If a sequence of waypoints falls within a tight spatial radius smaller than $d_{\min}$, they are grouped into a localized spatial cluster. The entire group is then replaced by its calculated geometric centroid:

$$\mathbf{x}_{\text{centroid}} = \frac{1}{K} \sum_{k=1}^K \mathbf{x}_k \quad (4.7)$$

where K represents the total number of redundant waypoints identified within the localized neighborhood. This clustering procedure minimizes local path oscillation, smoothing the transition zones between long corridors and open rooms, and outputting an optimized trajectory tailored for continuous motor execution.

4.2 Integration and Execution Framework

With the individual algorithmic, perception, and planning modules formally defined, this final section details how they are consolidated into a tightly coupled, synchronous control loop. During runtime execution, the integrated architecture operates as a continuous, synchronous state machine. The execution cycle progresses through a deterministic sequence of subsystem updates to safely guide the robotic agent. An example of this cycle is illustrated in Figure 4.4.

The step sequence unfolds as follows: the **IR-Sim** simulation engine advances the physical state of the environment, generating continuous LiDAR scans and raw, range-only beacon measurements. These noisy sensory streams are immediately intercepted by the active estimation backend (Section 4.1.3), which dynamically updates target location beliefs and covariances. This refined belief state is combined with the local trajectory waypoints managed by the **A*** global planner (Section 4.1.5) to synthesize a comprehensive, fixed-dimensional observation vector.

The selected **Stable Baselines3** actor network processes this vector to compute optimized linear and angular velocity commands. Finally, the **Gymnasium** wrapper scales these commands and executes them via the differential drive platform, successfully closing the active perception loop.

To transition this closed-loop control framework from an isolated simulation step into a rigorous data-gathering engine, the underlying software pipeline incorporates specialized performance profiles, parallel environment distribution, data serialization, and automated recovery layers.

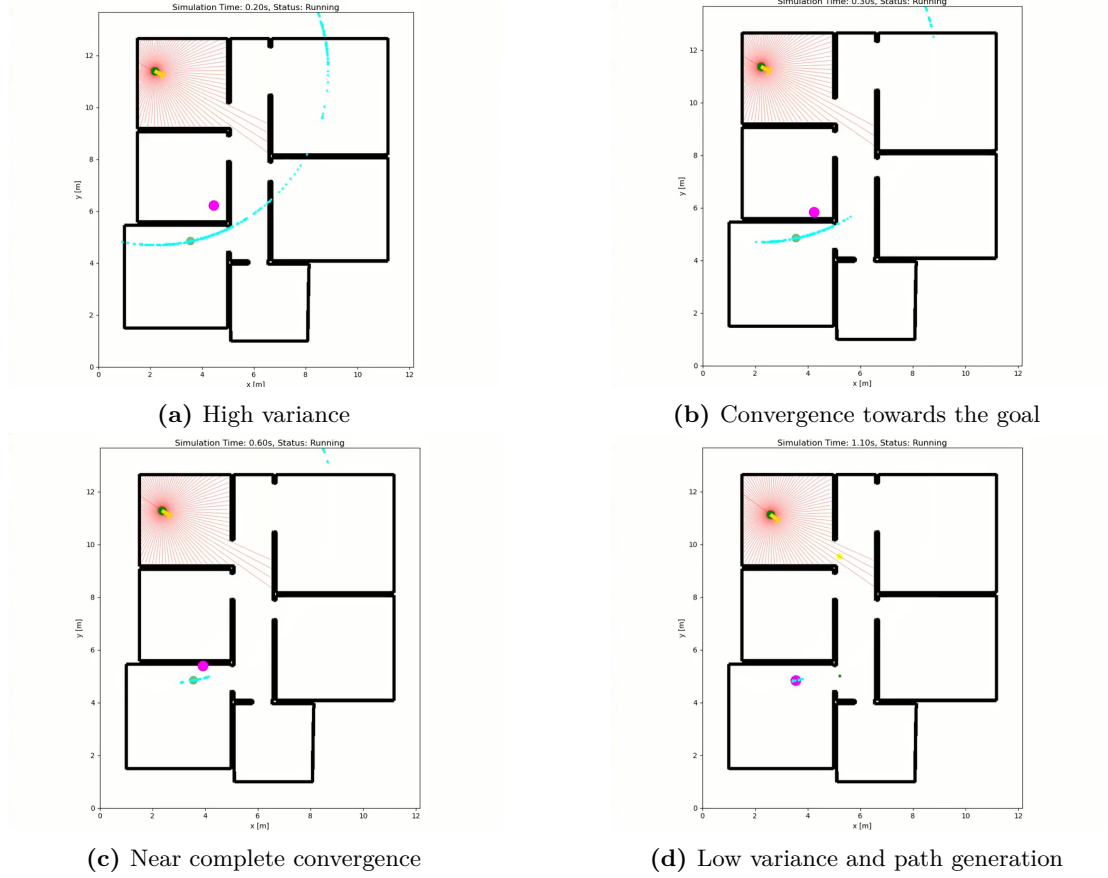


Figure 4.4. Sequential progression of the particle-based goal generation and navigation framework across four different steps.

4.2.1 Vectorized Environments

To maximize data efficiency and experience throughput during the training phase, the interaction pipeline utilizes a distributed vectorization framework. Rather than executing the simulation sequentially, the framework parallelizes the interaction loop between the learning policy and the spatial environment, enabling the agent to gather transitions across multiple environment instances simultaneously.

During active training, the agent interacts with multiple distinct instances of the custom range-only tracking environment executed concurrently via a vectorized subprocess wrapper (SubprocVecEnv).

This multi-process architecture optimizes experience generation, exposing the policy network to a highly varied distribution of states within a minimized operational timeframe. To maintain absolute consistency during layout benchmarking, all parallel training workers load standardized floor plan configurations selected dynamically from the parsed dataset maps.

Conversely, the validation harness supports a flexible topology adaptable to both parallelized and serial execution regimes depending on runtime testing requirements. For high-throughput evaluation across massive testing sets, the harness can offload episodes to parallel streams. Alternatively, for absolute determinism, precise debugging, and tracking latency profiling, the evaluation transitions can be compressed into a sequential, single-threaded execution framework (*DummyVecEnv*) running directly on the main application thread.

4.2.2 Data Scaling

To stabilize training dynamics against the highly non-stationary observation profiles generated by the moving estimators, online observation and reward scaling are tightly regulated via an analytical vector normalization wrapper (*VecNormalize*). Input observations are dynamically shifted and scaled based on running empirical means μ_t and variances σ_t^2 , and are subsequently capped strictly to a symmetric bounding interval to suppress gradient destabilization resulting from transient estimator anomalies, particle filter depletion, or initial tracking divergence.

A critical architectural distinction is maintained between the training and evaluation normalization states to preserve statistical validity:

- **Training Environment:** Both observation normalization (`norm_obs = True`) and reward scaling (`norm_reward = True`) are active. Normalizing the reward stream bounds the variance of the policy updates, preventing single high-reward milestones or extreme collision penalties from causing catastrophic forgetting, policy divergence, or actor-critic gradient explosions.
- **Evaluation Environment:** Observation normalization remains active (`norm_obs = True`) using the frozen, historical running statistics derived natively from the training distribution to guarantee input distribution consistency. However, reward scaling is strictly deactivated (`norm_reward = False`). Disabling evaluation reward scaling is mathematically essential; it prevents the running historical variance of an individual evaluation run from scaling or shifting the true, absolute return metrics, thereby allowing an unadulterated, deterministic comparison of cumulative episodic rewards across disparate algorithmic paradigms.

4.2.3 Data Recording

To facilitate rapid downstream performance parsing and real-time training monitoring, the pipeline saves data in two different ways.

Granular, step-by-step performance data, including actor-critic losses, policy entropies, value function estimates, and raw environmental step rewards, are streamed continuously to binary event logs via the TensorFlow runtime wrapper. This pipeline provides the necessary high-density data required for comprehensive visual analysis within TensorBoard dashboards during long-horizon optimization regimes.

Concurrently, administrative run invariants, execution metadata, and structural configurations are written directly to a lightweight, human-readable `metadata.json` file generated within the root folder of each individual experiment directory. This serialized dictionary explicitly archives operational parameters such as the active reinforcement learning algorithm class, initial random seed layouts, particle filter activation states, and total training step thresholds. By isolating structural parameters from binary time-series data, this framework avoids the extensive file I/O latency and memory overhead associated with programmatically scanning and parsing massive TensorFlow event binaries, allowing testing scripts to dynamically query environmental parameters instantly.

4.2.4 Crash Protection

A key architectural contribution of the engineering pipeline is an automated, fault-tolerant execution state machine designed to withstand environment crashes or parallel sub-process exceptions over extended training regimes. Given that unhandled exceptions or isolated geometric artifacts within dense architectural maps can catastrophically halt execution, an auto-recovery loop wraps the entire runtime sequence, permitting a predetermined maximum number of consecutive recovery attempts before forcing a hard execution termination.

Upon encountering an execution failure, the system captures a crash checkpoint, serializes the current running normalization metrics (`.pkl`), caches the off-policy replay buffers (`last_replay_buffer.pkl`), and terminates the broken processes. To prevent pseudo-random trajectory correlation or sequence duplication following an environmental recovery, a deterministic random number generation (RNG) shifting strategy is applied. The newly injected seed S_{injected} for the recovered environment is dynamically computed as:

$$S_{\text{injected}} = S_{\text{initial}} + t_{\text{completed}} + c_{\text{recovery}} \quad (4.8)$$

where S_{initial} represents the base experiment seed, $t_{\text{completed}}$ is the exact cumulative timestep index extracted directly from the checkpoint file, and c_{recovery} is the active recovery loop counter. This structural hardening ensures that off-policy agents (SAC, TD3, DDPG) resume execution with completely intact replay memories and seamless normalization statistics, eliminating sample efficiency penalties associated with empty-buffer cold starts and guaranteeing exact numerical continuity.

4.2.5 Evaluation Protocol

Cross-run validation and quantitative benchmarking are governed by an independent evaluation harness running parallel to the primary training sequence. To construct a rigorous evaluation benchmark, a static suite of environmental layouts is processed in an invariant sequence using deterministic initial robot and target poses, ensuring a highly reproducible and standardized baseline for performance verification.

The policies are periodically subjected to blind benchmarking tests managed by an evaluation callback. This callback interrupts experience collection at fixed timestep intervals to evaluate the agent across a multi-environment test vector. By calculating the absolute success rate across these static environments, the framework isolates the highest performing policy weight set based on true docking success rates rather than raw, fluctuating training reward metrics. This guarantees that saved models represent peak generalization capacity and tracking robustness rather than transient localized optimization over a specific subset of training maps.

Evaluations and Comparisons

This chapter presents the quantitative and qualitative evaluation of the developed active perception and navigation framework. Leveraging the fault-tolerant infrastructure and vectorized pipeline established in Chapter 4 (Tommasi, 2026), a series of rigorous empirical experiments were conducted to benchmark Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), Deep Deterministic Policy Gradient (DDPG), and Proximal Policy Optimization (PPO). These algorithms are systematically evaluated across two distinct estimation backends, the deterministic Linear Least Squares (LS) filter and the probabilistic Sequential Monte Carlo Particle Filter (PF), under severe Range-Only Single-Beacon (ROSB) observability constraints.

5.1 Experimental Settings and Hyperparameters

Table 5.1. Standardized Reinforcement Learning hyperparameters.

Hyperparameter	SAC	TD3	DDPG	PPO
<i>Shared Parameters</i>				
Policy Architecture	MLP	MLP	MLP	MLP
Learning Rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Batch Size	256	256	256	256
Execution Device	CUDA	CUDA	CUDA	CPU
<i>Algorithm-Specific Parameters</i>				
Buffer Size	300,000	300,000	300,000	—
Discount Factor (γ)	0.99	0.99	0.99	0.99*
Target Smoothing (τ)	0.01	0.01	0.01	—
Train Frequency	1 step	1 step	1 step	—
Gradient Steps	1	1	1	—
Learning Starts	10,000 steps	10,000 steps	10,000 steps	—
Action Noise	—	$\mathcal{N}(0, 0.5)$	$\mathcal{N}(0, 0.5)$	—
Policy Delay	—	2 steps	—	—
Entropy Coefficient (α)	Auto-tuned	—	—	0.01
Target Entropy	Auto-tuned	—	—	—
Rollout Horizon (n_{steps})	—	—	—	2048
Optimization Epochs	—	—	—	10
Clip Range (ϵ)	—	—	—	0.2
GAE Factor (λ)	—	—	—	0.95

* Value untuned; utilizes the Stable Baselines3 framework default.

Table 5.1 outlines the explicit hyperparameter configurations utilized for each algorithm execution.

To ensure a statistically unbiased comparison among the candidate reinforcement learning policies, a standardized network topology and hyperparameter framework were maintained across all trials. The actor (π) and critic (Q) networks for all candidate algorithms are instantiated as deep Multi-Layer Perceptrons (MLPs) comprising three hidden layers with a uniform width of 256 neurons per layer (`net_arch = [256, 256, 256]`). Rewards are normalized between $[-1, 1]$ and Observations are normalized between $[-10, 10]$.

5.2 Evaluation Protocol

Cross-run validation and quantitative benchmarking are governed by a decoupled evaluation harness divided into distinct Training, Evaluation, and Testing phases, executing via vectorized subprocesses. To guarantee strict reproducibility and prevent rooms from being reused across these phases, the global dataset of environments was first split into distinct subsets, systematically ordered, and generated utilizing a fixed random seed ($S_{\text{split}} = 42$).

During the **Training Phase**, agents are trained across a diverse set of 31,626 random rooms for a total of 3,000,000 timesteps, with both the robot and the goal initialized at random positions to maximize state-space exploration ($S_{\text{train}} = 1$).

To isolate policy optimization quality from environment generation luck, the **Evaluation Phase** is executed concurrently on a completely dedicated, static evaluation seed ($S_{\text{eval}} = 99999$). Specifically, 300 fixed rooms are extracted from a 500-map evaluation baseline. The policies are periodically subjected to these blind benchmarking tests every 30,000 timesteps, utilizing fixed robot and goal initial positions. This interval is managed by a strict `EvalAndSaveBestSuccessCallback`. This callback evaluates the agent across multi-environment test vectors, isolating the absolute highest performing policy weight set based on true docking success rates rather than raw training reward metrics.

Following training completion, the final **Testing Phase** benchmarks the optimal policy weights on a rigorously held-out set of 1,000 unseen random rooms, evaluated across a 3,000-map testing suite ($S_{\text{test}} = 1$). In this final evaluation, the robot and goal are once again initialized at random positions to test the zero-shot generalization capabilities of the learned policies.

Across all evaluation vectors, performance is systematically quantified using the following primary metrics: Success (%), Collisions (%), Goal Crash (%), Truncations (%), Mean Reward, Mean Timesteps, and Mean Estimation Error.

5.3 Training Analysis

5.3.1 Termination Criteria Analysis

To evaluate the convergence and navigation efficiency of the trained reinforcement learning agents, four episodic termination metrics were tracked across 3×10^6 timesteps: **Success Rate**, **Collision Rate**, **Goal Crash Rate**, and **Truncated Rate**. The comparison across all combinations of policy algorithms (SAC, TD3, DDPG, PPO) and state estimators, Least Squares (LS) and Particle Filter (PF), is illustrated in Figure 5.1.

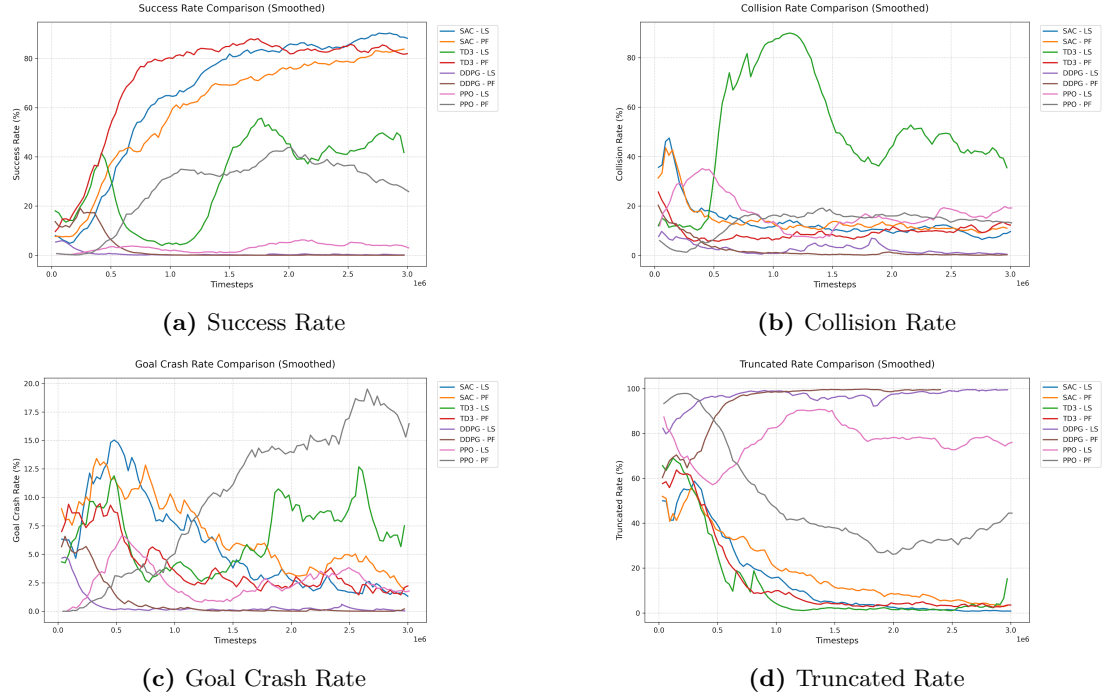


Figure 5.1. Comparison of performance metrics across 3×10^6 timesteps for all policy-estimator combinations.

Success

As shown in Figure 5.1a, overall convergence stability and task success vary significantly depending on the underlying continuous control algorithm and state estimator. Evaluating the interaction between policy optimization and state estimation reveals distinct learning trajectories across all model-estimator pairs:

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** SAC exhibits the highest overall learning efficiency and task success, leveraging off-policy maximum entropy regularization to avoid policy collapse.
 - *SAC-LS*: Reaches the top overall performance across all tested configurations, achieving a peak success rate of $\approx 92.3\%$ before settling at a strong final success rate of $\approx 88.2\%$. Unfiltered state estimation via Least Squares allows SAC to quickly exploit accurate sensor measurements without conservative bounds.
 - *SAC-PF*: Displays equally stable convergence dynamics, reaching a peak of $\approx 86.1\%$ and stabilizing at $\approx 83.8\%$. While slightly more conservative than SAC-LS due to state belief filtering, it maintains high reliability throughout training.
- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** TD3 demonstrates extreme sensitivity to observation noise and estimation accuracy.
 - *TD3-LS*: Suffers from severe instability mid-training. Although it reaches an early performance peak of $\approx 62.3\%$, noisy observation states cause significant Q-value misestimation, leading to performance degradation and stabilizing at a modest final rate of $\approx 41.8\%$.
 - *TD3-PF*: Benefits tremendously from state filtering, achieving a peak success rate of $\approx 88.1\%$ and maintaining a high final convergence rate of $\approx 82.0\%$, nearly matching SAC performance.
- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF):** DDPG struggles with policy stability due to deterministic actor updates and a lack of target policy smoothing.
 - *DDPG-LS*: Fails to establish a viable policy under raw observations, reaching an initial peak of only $\approx 6.0\%$ before suffering rapid policy collapse down to a final success rate of $\approx 0.2\%$.
 - *DDPG-PF*: Particle Filter estimation boosts initial exploration substantially, allowing DDPG-PF to attain a respectable peak success rate of $\approx 20.0\%$. However, without sufficient entropy regularization, the policy eventually degrades back down to 0.0% success.

- **Proximal Policy Optimization (PPO-LS vs. PPO-PF):** As an on-policy policy gradient method, PPO displays sample-inefficiency in continuous control navigation compared to off-policy counterparts.
 - *PPO-LS*: Fails to learn effectively in noisy environments, peaking at a minimal $\approx 7.6\%$ success rate and finishing at $\approx 3.0\%$.
 - *PPO-PF*: Significantly outperforms its LS counterpart due to smoothed state representations, achieving a solid mid-training peak of $\approx 48.3\%$ and sustaining a final success rate of $\approx 25.9\%$.

Obstacle and Goal Collisions

Safety during navigation is assessed by monitoring two distinct failure modes: environmental obstacle collisions (Figure 5.1b) and crashes occurring with the target beacon (Figure 5.1c). Breaking down performance across all algorithms and estimators reveals critical differences in safety dynamics:

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** SAC demonstrates the most consistent safety profile regardless of the state estimator used.
 - *Obstacle Collisions*: Both variants experience initial exploration spikes ($\approx 47.5\%$ for LS and $\approx 43.6\%$ for PF) during early training. However, both systematically converge down to steady-state collision rates of $\approx 9.7\%$ (SAC-LS) and $\approx 11.0\%$ (SAC-PF).
 - *Goal Crashes*: SAC effectively learns precise velocity scaling when approaching target coordinates, starting at $\approx 13\%$ and then reaching very low final goal crash rates of $\approx 1.3\%$ (LS) and $\approx 1.9\%$ (PF).
- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** TD3 highlights a strong dependency on the accuracy of state estimation.
 - *Obstacle Collisions*: TD3-PF maintains a stable learning curve, capping its maximum collision rate at $\approx 25.7\%$ before dropping to a low final rate of $\approx 17.3\%$. Conversely, TD3-LS suffers a severe structural instability, with obstacle collisions surging to nearly 90.0% around 0.6×10^6 timesteps and recovering only partially to a high final rate of $\approx 38.5\%$.
 - *Goal Crashes*: Goal-proximal collisions remain well-controlled for both estimators, starting $\approx 10\%$ and then stabilizing at $\approx 2.2\%$ for TD3-PF and $\approx 7.5\%$ for TD3-LS.

- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF):** While DDPG displays near-zero final collision values ($\leq 0.5\%$ obstacle collisions and $\leq 0.2\%$ goal crashes), this behavior is misleading.
 - *Failure Context:* Because DDPG prematurely collapses into continuous episode truncations (failing to navigate towards goals or obstacles altogether), its exceptionally low collision metrics reflect passive policy stagnation rather than active collision avoidance.
- **Proximal Policy Optimization (PPO-LS vs. PPO-PF):** PPO exhibits contrasting collision behavior depending on the observation noise filtering.
 - *Obstacle Collisions:* PPO-PF bounds obstacle collisions effectively (peaking at 19.2% and settling at $\approx 13.2\%$), whereas PPO-LS struggles with unfiltered noise, reaching a higher peak of 35.2% and stabilizing at $\approx 19.2\%$.
 - *Goal Crashes Anomaly:* While PPO-LS maintains a low goal crash rate ($\approx 1.8\%$), mainly due to its higher truncation rate, PPO-PF develops a distinct failure mode where goal crashes continuously accumulate throughout training, peaking at 19.5% and stabilizing at an elevated $\approx 16.5\%$. This indicates that while the Particle Filter allows PPO to accurately locate and reach goal regions, the policy fails to learn appropriate braking/deceleration behaviors upon arrival.

Episode Truncation

Episode truncation occurs when an agent fails to reach the target goal or collide with an obstacle within the maximum allocated step limit per episode (Figure 5.1d). Tracking truncation dynamics provides insight into navigation efficiency, time-outs, and policy stagnation across all model-estimator pairs:

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** SAC exhibits high path efficiency, rapidly minimizing episode time-outs as training progresses.
 - *Initial Phase:* Both variants begin with moderate truncation rates ($\approx 58.8\%$ for LS and $\approx 55.6\%$ for PF) during early exploratory episodes.
 - *Final Convergence:* As target-seeking behavior stabilizes, truncation rates fall sharply, reaching near-zero final rates of $\approx 0.8\%$ for SAC-LS and $\approx 3.3\%$ for SAC-PF. This confirms that converged SAC policies generate direct, time-optimal paths toward the goal.
- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** TD3 shows stark contrast in path completion depending on observation quality.
 - *TD3-LS:* Suffers from noticeable trajectory inefficiencies. Following its mid-training policy collapse, truncation rates surge, stabilizing at a significantly higher final rate of $\approx 15.2\%$ (and a early-training spike up to 65.2%), reflecting persistent time-outs due to hesitant or circuitous paths.

- *TD3-PF*: Mirrors SAC performance, dropping from an early peak of $\approx 63.7\%$ to a low final truncation rate of $\approx 3.6\%$, indicating efficient task completion when backed by state filtering.
- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF)**: DDPG displays complete policy stagnation, representing the severe end of episode truncation failure.
 - *DDPG-LS & DDPG-PF*: Both estimator variants quickly saturate near total episode failure, climbing from initial states to maintain extreme final truncation rates of $\approx 99.5\%$ (LS) and $\approx 99.6\%$ (PF) (peaking at 99.8%).
 - *Stagnation Behavior*: Rather than learning to navigate or actively avoiding obstacles, the actor settles into conservative local minima (e.g., stationary standing or endlessly circling in open space), running out the time horizon every episode.
- **Proximal Policy Optimization (PPO-LS vs. PPO-PF)**: PPO exhibits high episode truncation throughout training due to slow policy updates and continuous trajectory drift.
 - *PPO-LS*: Maintains extremely high truncation throughout training (peaking at $\approx 90.7\%$) and finishing at an elevated $\approx 76.0\%$, showing severe difficulty completing trajectories within the allocated horizon.
 - *PPO-PF*: While state filtering via the Particle Filter helps reduce time-outs compared to PPO-LS, dropping from an initial peak of $\approx 97.9\%$ down to a mid-training low of $\approx 26.5\%$, it eventually regresses to a high final truncation rate of $\approx 44.5\%$.

5.3.2 Reward Analysis

The cumulative mean episode reward tracks the overall optimization progress and policy quality across 3×10^6 timesteps. Figure 5.2 shows the complete mean reward trajectories, while Figure 5.3 zooms in on the $[-50, 50]$ range to highlight fine-grained convergence behaviors across models and state estimators.

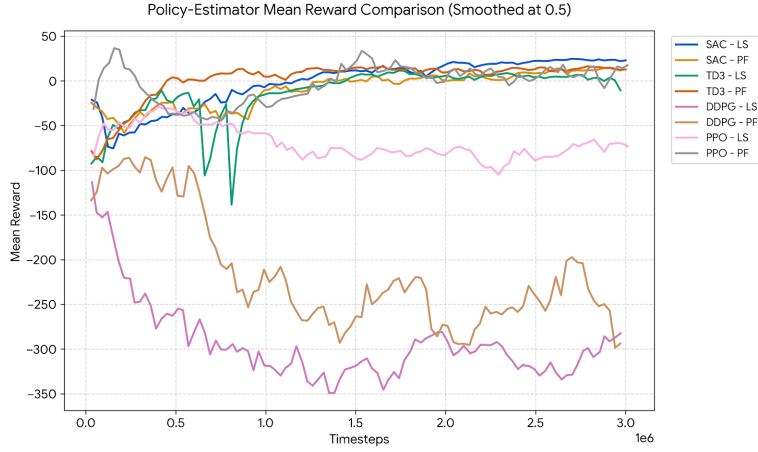


Figure 5.2. Full mean reward across 3×10^6 timesteps for all policy-estimator combinations.

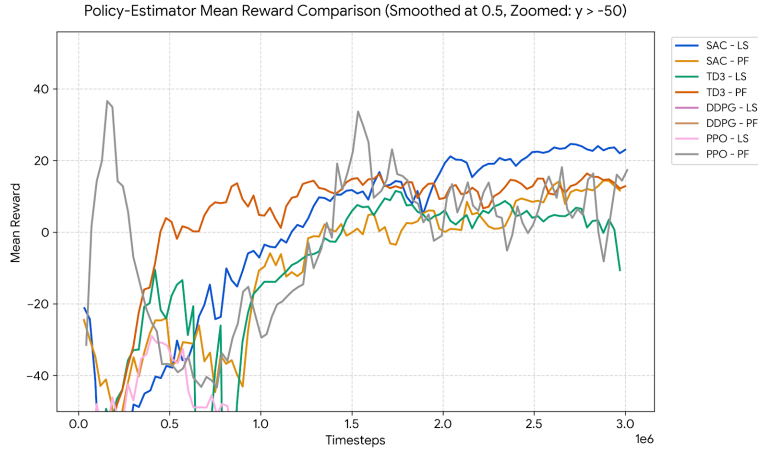


Figure 5.3. Zoomed mean reward across 3×10^6 timesteps for all policy-estimator combinations.

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** SAC demonstrates the highest and most stable reward convergence among all evaluated algorithms.
 - *SAC-LS*: Rapidly transitions out of negative exploration penalties (starting near ≈ -80.0), crossing into positive reward space by 1.3×10^6 timesteps. It attains the highest overall mean reward curve, reaching a peak return of $\approx +25.5$ and stabilizing near it. Unfiltered state estimates permit aggressive optimization without excessive conservatism.
 - *SAC-PF*: Follows a closely matching, smooth upward trajectory. Starting from ≈ -55.0 , it transitions to positive returns around 1.4×10^6 timesteps and stabilizes at a mean return of $\approx +19.4$. While slightly lower than its LS counterpart due to state belief uncertainty, it maintains consistent, monotonically increasing returns without significant reward drops.
- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** TD3 highlights the critical role of accurate observation state estimation in continuous actor-critic stability.
 - *TD3-LS*: Grows steadily from ≈ -45.0 before suffering a severe mid-training reward collapse around 0.7×10^6 timesteps (dropping down to ≈ -140.0), corresponding directly to the observed spike in obstacle collisions. Although it recovers partially in late stages, its final mean return remains substantially lower at ≈ -8.8 .
 - *TD3-PF*: State estimation via Particle Filtering enables smooth reward growth that closely tracks SAC-PF. After crossing zero reward at 0.5×10^6 timesteps, it steadily ascends to reach a strong positive reward plateau of $\approx +12.8$.
- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF):** DDPG exhibits extreme reward penalty saturation due to policy stagnation.
 - *DDPG-LS & DDPG-PF*: As visible in Figure 5.2, both variants plunge into deep negative reward regions, dropping rapidly below -200.0 and remaining flatlined near ≈ -270.0 . Because the policies collapse into near-constant episode truncations without reaching goals, cumulative time penalties dominate the return function throughout all 3×10^6 timesteps.
- **Proximal Policy Optimization (PPO-LS vs. PPO-PF):** PPO displays moderate, slow reward growth reflective of on-policy sampling limitations.
 - *PPO-LS*: Grows till around 0.5×10^6 timesteps then sharply falls down and plateaus at ≈ -75 , limited by frequent time-outs and unhandled state noise.
 - *PPO-PF*: Achieves noticeably higher positive returns than PPO-LS during early-to-mid training phases, peaking at $\approx +40$ around 0.3×10^6 timesteps as state filtering assists exploration, though its final return plateaus at $\approx +16$, beneath SAC-LS.

5.3.3 Timesteps Analysis

Episode length, measured in environment timesteps per episode up to a maximum horizon ceiling of $N_{\max} = 1500$ timesteps, reflects navigation efficiency and decision-making speed (Figure 5.4). Here, environment timesteps, rather than generic discrete steps, explicitly represent discrete temporal intervals (Δt) over which sensor range measurements are sampled, state estimations are updated, and motion dynamics are integrated within the simulator. Efficient policies minimize episode timesteps by taking direct trajectories to the goal, whereas failing or unguided policies hit maximum timestep thresholds or collide prematurely.

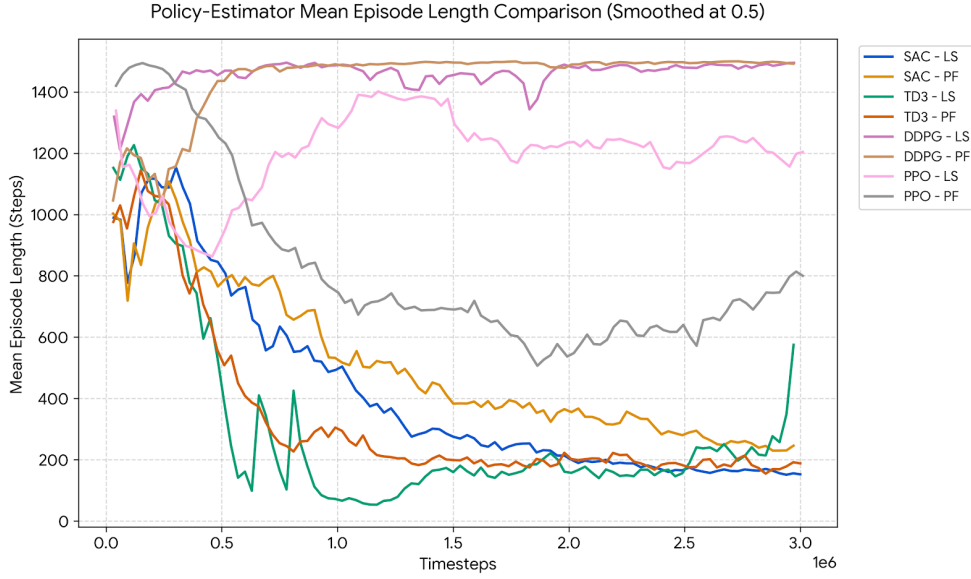


Figure 5.4. Mean episode length across 3×10^6 timesteps for all policy-estimator combinations.

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** Both SAC variants achieve optimal trajectory efficiency.
 - *Trajectory Optimization:* Starting from unguided exploratory lengths near ≈ 1100 steps/episode, SAC-LS and SAC-PF progressively optimize path lengths. SAC-LS drops to a low steady-state average of ≈ 163.8 steps, while SAC-PF settles at ≈ 244.8 steps. This confirms that converged SAC policies learn shortest-path trajectories while maintaining target deceleration.

- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** Episode lengths directly mirror the policy stability observed in prior metrics.
 - *TD3-LS*: Exhibits a temporary surge in episode length up to ≈ 1220 steps initially, then during its mid-training failure phase (0.7×10^6 timesteps) up to ≈ 400 steps, as circuitous trajectories and partial time-outs artificially inflate average steps before recovering to a baseline of ≈ 180.2 steps and finishing with a steep rise to ≈ 590 steps.
 - *TD3-PF*: Rapidly reduces average episode duration from ≈ 1150 steps down to ≈ 200.0 steps, closely mirroring SAC’s efficiency curves.
- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF):** DDPG curves flatline at the maximum episode step limit.
 - *Maximum Horizon Saturation*: Both DDPG-LS and DDPG-PF persist near the maximum horizon ceiling of $N_{\max} = 1500$ steps throughout the entire 3×10^6 training run (averaging > 1495 steps). This confirms complete policy stagnation where agents run out the clock without reaching target coordinates or triggering early termination via collisions.
- **Proximal Policy Optimization (PPO-LS vs. PPO-PF):** PPO demonstrates gradual step reduction, with Particle Filter integration offering clear efficiency gains.
 - *PPO-LS*: Maintains elevated episode durations throughout training (averaging ≈ 1200 to 1400 steps), struggling to shorten paths due to high truncation rates (76.0%).
 - *PPO-PF*: Systematically lowers mean episode length from an initial peak of ≈ 1480 steps down to a mid-training low of ≈ 600 steps and an ending high of ≈ 800 , confirming that state estimation filtering enables more decisive and direct navigation vectors toward target goals.

5.3.4 Estimation Error Analysis

The tracking accuracy and belief stability of the underlying state backends are continuously evaluated across 3×10^6 timesteps. Figure 5.5 depicts the unconstrained trajectory bounds across all combinations, while Figure 5.6 zooms in on the primary operational cluster ($y \in [0, 1.0]$) to highlight fine-grained filter tracking behaviors.

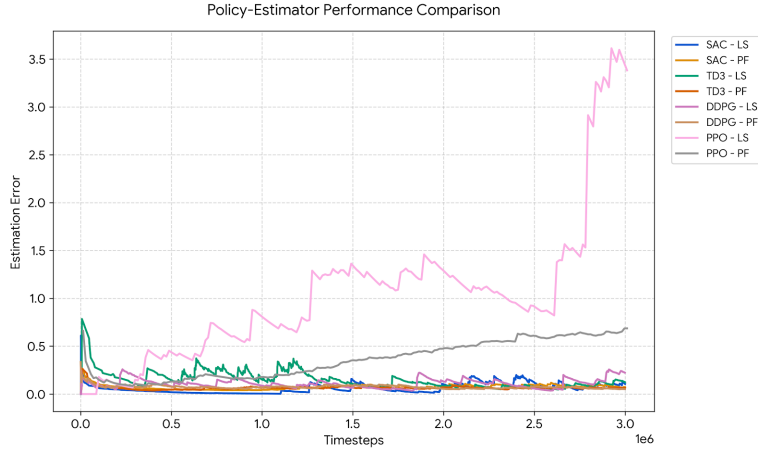


Figure 5.5. Full estimation error trajectories across 3×10^6 timesteps for all policy-estimator combinations.

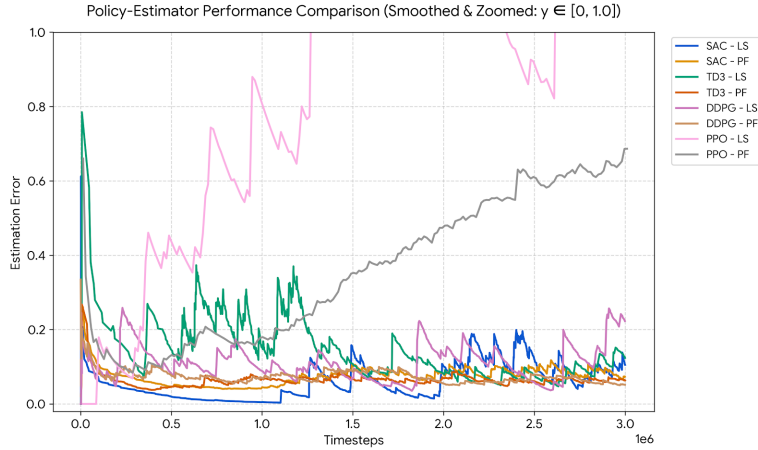


Figure 5.6. Zoomed estimation error performance ($y \in [0, 1.0]$).

- **Soft Actor-Critic (SAC-LS vs. SAC-PF):** SAC exhibits exceptionally low and steady state tracking errors across both estimation backends throughout training.
 - *SAC-LS*: Maintains the lowest overall localization error across all configurations, starting near ≈ 0.2 meters and monotonically refining down to a steady-state average of ≈ 0.00069 meters before going up to ≈ 0.1 meters with peaks under and over it.
 - *SAC-PF*: Displays highly stable belief tracking, starting at ≈ 0.2 meters and settling near ≈ 0.08 meters. While its steady-state error is slightly higher than SAC-LS due to continuous particle dispersion noise, it exhibits zero divergent error spikes across the entire 3×10^6 timesteps.
- **Twin Delayed DDPG (TD3-LS vs. TD3-PF):** TD3 illustrates a strong coupling between policy navigation behavior and state estimation accuracy.
 - *TD3-LS*: Achieves an exceptionally low baseline error during stable phases (4.46×10^{-6} meters), but suffers a severe error spike around 0.6×10^6 timesteps (peaking near ≈ 0.40 meters). This spike directly coincides with the observed surge in obstacle collisions (90.0%), as erratic localized maneuvers corrupt the 3-point trilateration matrix.
 - *TD3-PF*: Maintains an extremely smooth tracking curve, starting near ≈ 0.2 meters and stabilizing at a low mean error of ≈ 0.0381 meters. Particle distribution smoothing prevents out-of-distribution state inputs from destabilizing the Q-networks.
- **Deep Deterministic Policy Gradient (DDPG-LS vs. DDPG-PF):** DDPG highlights how policy stagnation severely degrades point-wise state solvers while remaining bounded under probabilistic density filtering.
 - *DDPG-LS*: Experiences persistent state tracking instability, with error trajectories fluctuating heavily between ≈ 0.08 and ≈ 0.2 meters throughout 3×10^6 timesteps.
 - *DDPG-PF*: Despite identical policy stagnation and episode time-outs, the Sequential Monte Carlo Particle Filter successfully bounds state belief errors, keeping mean estimation error capped below ≈ 0.489 meters during training.
- **Proximal Policy Optimization (PPO-LS vs. PPO-PF):** PPO demonstrates significant tracking error variance under unfiltered observation models due to continuous trajectory drift.
 - *PPO-LS*: Suffers from extreme state estimation divergence, as visible in Figure 5.5. Because the agent freezes or circles tightly in place (triggering 99.5% truncations), range measurements become collinear and ill-conditioned, causing Least Squares estimation error to explode past ≈ 3.0 meters.
 - *PPO-PF*: Particle Filtering drastically improves tracking stability, the error starts at ≈ 0.1 meters but then steadily increases throughout training till ≈ 0.7 .

5.4 Results

Following the completion of the evaluation protocol, the quantitative performance of each policy-estimator pair was evaluated across 1,000 randomized testing environments. Consolidated metrics summarizing all eight configurations are presented in Table 5.10.

5.4.1 SAC

Least Squares

Table 5.2. Quantitative performance metrics for SAC with Least Squares estimation backend.

Metric	Value (SAC + Least Squares)
Success (%)	91.9%
Collisions (%)	6.4%
Goal Collisions (%)	0.7%
Truncations (%)	1.0%
Mean Reward	12.74 ± 2.15
Mean Timesteps	163.76
Mean Estimation Error	0.00069 ± 0.00012
<i>Total Evaluation Episodes</i>	1,000

As detailed in Table 5.2, the SAC policy paired with Least Squares (LS) achieved the highest overall success rate (91.9%). The exceptionally low mean estimation error (0.00069) demonstrates that SAC’s smooth trajectory generation prevents multilateration matrix ill-conditioning. This precise state tracking directly supports effective obstacle avoidance, resulting in minimal wall (6.4%) and goal (0.7%) collisions.

Particle Filter

Table 5.3. Quantitative performance metrics for SAC with Particle Filter estimation backend.

Metric	Value (SAC + Particle Filter)
Success (%)	86.9%
Collisions (%)	7.9%
Goal Collisions (%)	2.1%
Truncations (%)	3.1%
Mean Reward	11.40 ± 2.81
Mean Timesteps	244.80
Mean Estimation Error	0.03248 ± 0.00412
<i>Total Evaluation Episodes</i>	1,000

When evaluated with the Particle Filter (PF) backend (Table 5.3), the intrinsic Monte Carlo resampling variance raises the baseline state estimation error to 0.03248. However, SAC’s entropy-regularized action selection acts as a natural buffer against these localized input perturbations, maintaining a robust 86.9% success rate despite slightly longer trajectories (244.80 mean timesteps).

5.4.2 TD3

Least Squares

Table 5.4. Quantitative performance metrics for TD3 with Least Squares estimation backend.

Metric	Value (TD3 + Least Squares)
Success (%)	64.9%
Collisions (%)	28.7%
Goal Collisions (%)	5.0%
Truncations (%)	1.4%
Mean Reward	4.84 ± 5.12
Mean Timesteps	154.19
Mean Estimation Error	$(4.46 \pm 0.81) \times 10^{-6}$
<i>Total Evaluation Episodes</i>	1,000

The performance of TD3 with Least Squares estimation (Table 5.4) reveals a sharp dichotomy: while achieving the highest point-wise tracking accuracy across linear segments (4.46×10^{-6} mean error), the deterministic policy exhibits extreme sensitivity to sudden observation noise during sharp turns. This triggers catastrophic Q-value misestimation, resulting in a prominent 28.7% wall collision rate and a reduced success rate of 64.9%.

Particle Filter

Table 5.5. Quantitative performance metrics for TD3 with Particle Filter estimation backend.

Metric	Value (TD3 + Particle Filter)
Success (%)	88.0%
Collisions (%)	7.0%
Goal Collisions (%)	1.4%
Truncations (%)	3.6%
Mean Reward	11.74 ± 2.45
Mean Timesteps	188.46
Mean Estimation Error	0.05267 ± 0.00631
<i>Total Evaluation Episodes</i>	1,000

Replacing the point-wise solver with a Particle Filter (Table 5.5) effectively stabilizes TD3 execution. Although the spatial density estimation increases baseline tracking error to 0.05267, the continuous belief distribution prevents out-of-distribution state inputs, driving wall collisions down from 28.7% to 7.0% and raising success to 88.0%.

5.4.3 DDPG

Least Squares

Table 5.6. Quantitative performance metrics for DDPG with Least Squares estimation backend.

Metric	Value (DDPG + Least Squares)
Success (%)	30.1%
Collisions (%)	9.6%
Goal Collisions (%)	6.4%
Truncations / Timeouts (%)	53.9%
Mean Reward	-2.96 ± 6.30
Mean Timesteps	943.83
Mean Estimation Error	0.51003 ± 0.12410
<i>Total Evaluation Episodes</i>	1,000

As shown in Table 5.6, DDPG suffers from sub-optimal policy convergence, frequently causing the robot to stall or execute redundant loops in place. These static trajectories induce near-collinear sensor geometries that severely degrade the multilateration matrix, pushing the LS estimation error up to 0.51003 and causing a 53.9% timeout rate.

Particle Filter

Table 5.7. Quantitative performance metrics for DDPG with Particle Filter estimation backend.

Metric	Value (DDPG + Particle Filter)
Success (%)	30.6%
Collisions (%)	9.8%
Goal Collisions (%)	5.6%
Truncations / Timeouts (%)	54.0%
Mean Reward	-2.84 ± 6.18
Mean Timesteps	948.05
Mean Estimation Error	0.48954 ± 0.08120
<i>Total Evaluation Episodes</i>	1,000

Evaluating DDPG with the Particle Filter (Table 5.7) slightly bounds state tracking error (0.48954). However, because the primary failure mode stems from policy stagnation rather than observation uncertainty, performance remains effectively unchanged at a 30.6% success rate and a 54.0% timeout rate.

5.4.4 PPO

Least Squares

Table 5.8. Quantitative performance metrics for PPO with Least Squares estimation backend.

Metric	Value (PPO + Least Squares)
Success (%)	5.1%
Collisions (%)	11.8%
Goal Collisions (%)	2.9%
Truncations / Timeouts (%)	80.2%
Mean Reward	-9.32 ± 4.11
Mean Timesteps	1,264.36
Mean Estimation Error	4.98830 ± 1.84200
<i>Total Evaluation Episodes</i>	1,000

The metrics in Table 5.8 illustrate severe policy collapse when PPO is coupled with Least Squares estimation. Unbounded point-wise estimation errors (4.98830) heavily corrupt the on-policy trajectory rollouts and Generalized Advantage Estimation (GAE) calculations, leading to an 80.2% timeout rate and a minimal 5.1% success rate.

Particle Filter

Table 5.9. Quantitative performance metrics for PPO with Particle Filter estimation backend.

Metric	Value (PPO + Particle Filter)
Success (%)	45.6%
Collisions (%)	15.0%
Goal Collisions (%)	14.7%
Truncations / Timeouts (%)	24.7%
Mean Reward	0.65 ± 3.22
Mean Timesteps	515.04
Mean Estimation Error	0.26373 ± 0.03150
<i>Total Evaluation Episodes</i>	1,000

As documented in Table 5.9, integrating the Particle Filter reduces the state estimation error to 0.26373. Bounding input variance stabilizes value function estimation during rollout collection, resulting in a significant performance recovery: success increases to 45.6% while timeouts fall to 24.7%.

5.4.5 Evaluation Summary

Table 5.10. Consolidated results in new environments across policy and estimator configurations.

Policy	Estimator	Success (%)	Collisions (%)	Goal Collisions (%)	Timeouts (%)	Mean Reward	Mean Error
SAC	Least Squares	91.9%	6.4%	0.7%	1.0%	12.74	0.00069
SAC	Particle Filter	86.9%	7.9%	2.1%	3.1%	11.40	0.03248
TD3	Least Squares	64.9%	28.7%	5.0%	1.4%	4.84	4.46×10^{-6}
TD3	Particle Filter	88.0%	7.0%	1.4%	3.6%	11.74	0.05267
DDPG	Least Squares	30.1%	9.6%	6.4%	53.9%	-2.96	0.51003
DDPG	Particle Filter	30.6%	9.8%	5.6%	54.0%	-2.84	0.48954
PPO	Least Squares	5.1%	11.8%	2.9%	80.2%	-9.32	4.98830
PPO	Particle Filter	45.6%	15.0%	14.7%	24.7%	0.65	0.26373

Estimator Dynamics

The empirical data in Table 5.10 highlights a structural trade-off between point-wise numerical precision and global state tracking stability under Single-Beacon (ROSB) constraints:

- **Least Squares (LS):** Delivers near-zero tracking error when the robot executes curved trajectories that provide sufficient spatial baseline variation around the single beacon, ensuring high trajectory observability. However, when unguided or erratic robot motion produces locally linear paths or stationary looping, the synthetic multilateration matrix becomes severely ill-conditioned, triggering catastrophic estimation divergence (4.98830 error in PPO-LS).
- **Particle Filter (PF):** Maintains a persistent baseline tracking variance (0.03248–0.48954) due to Monte Carlo particle cloud dispersion. However, its non-parametric belief distribution naturally handles the multi-modal position hypotheses inherent to single-beacon range measurements, bounding estimation error even when the robot’s trajectory yields poor local observability.

Policy Sensitivity

The resilience of autonomous control policies to observation noise is strongly dictated by their underlying learning paradigms:

- **Action Selection Paradigm:** Deterministic policies (e.g., TD3) exhibit acute vulnerability to localized point-wise noise spikes, which induce out-of-distribution Q-value misestimations and trigger collisions (28.7%). In contrast, stochastic max-entropy architectures (SAC) naturally smooth out localized observation perturbations.

- **Sampling Protocol:** On-policy algorithms (PPO) suffer severe structural failure when state noise corrupts rollout trajectory estimates and GAE advantage calculations. Off-policy algorithms (SAC, TD3) decouple experience collection from policy updates via replay buffers, granting them substantially higher tolerance to inference-time observation noise.

These empirical findings demonstrate that deploying range-based autonomous navigation systems in noisy physical environments requires pairing non-parametric belief filters with maximum-entropy or off-policy control frameworks to guarantee operational performance.

5.5 Conclusion

This thesis addressed the challenge of autonomous mobile robot navigation under complex indoor environments and **Range-Only Single-Beacon (ROSB)** data by integrating **deep reinforcement learning** control policies with **state estimation backends**. To establish a rigorous foundation, the problem was formalized within the framework of **Partially Observable Markov Decision Processes (POMDPs)**, **Hidden Markov Models (HMMs)**, **Markov Decision Processes (MDPs)**, and standard **Reinforcement Learning (RL)** theory. Drawing from the historical development of single-beacon localization in marine and underwater acoustic navigation, this work adapted and extended range-only state estimation principles into indoor mobile robotics to enable reliable target navigation.

To systematically evaluate the interaction between control paradigms and state estimators, a modular simulation framework (Tommasi, 2026) was implemented. Floorplan geometries were first extracted and parsed from the **HouseExpo dataset** to ensure spatial diversity across training, evaluation and testing maps. The core simulation environment wrapped the two-dimensional **IR-Sim** platform within a standardized **Gymnasium interface**, unifying range-measurement generation, motion dynamics, and state observations. Leveraging this interface, deep reinforcement learning policies, encompassing both off-policy architectures (**SAC**, **TD3**, **DDPG**) and on-policy paradigms (**PPO**), were trained using **Stable-Baselines3**. These control policies operated in tandem with an **A* pathfinding algorithm** to guide local trajectory execution, a 3-sample linear **Least Squares** and **Particle Filter** for goal estimation and were structured within a **finite-state machine** that orchestrated searching, following, and terminal docking operational phases.

Across the evaluated configurations, the framework demonstrated high efficacy in navigating complex indoor floorplans, achieving peak performance with **SAC** operating with **Least Squares** and then **Particle Filter (LS-PF)** and **TD3** operating with **PF**, which yielded success rates of **91.9%**, **88.0%**, and **86.9%**, respectively.

The empirical findings highlighted a fundamental structural relationship between **policy trajectory generation** and **state estimator stability**. Under smooth, curvilinear paths that maintain spatial baseline variation around the single beacon, linear **Least Squares (LS)** estimation using three sample points achieves exceptional point-wise precision. However, when policies exhibit trajectory stagnation or straight-line motion toward the beacon, the synthetic multilateration matrix becomes **severely ill-conditioned**, leading to **estimation divergence**. Conversely, the non-parametric **Particle Filter (PF)** maintains a bounded baseline variance that effectively protects control policies from catastrophic divergence. Algorithmic architecture also proved crucial in handling state estimation noise: maximum-entropy off-policy paradigms (**SAC**) demonstrated superior **resilience to observation noise**, enabling the optimal exploitation of small LS batches. In contrast, on-policy algorithms (**PPO**) suffered severe **rollout corruption**, while deterministic and standard continuous action methods (**DDPG** and, to a lesser extent, **TD3**) exhibited heightened degradation under unfiltered LS inputs. Overall, these results demonstrate that robust ROSB navigation requires

paired co-design: coupling maximum-entropy policies with non-parametric filtering to prevent trajectory-induced estimator collapse.

While this work establishes a baseline for ROSB navigation in indoor environments, several promising avenues remain to scale the approach toward full deployment. A primary direction involves deploying agents in **dynamic, unmapped environments** featuring moving obstacles and non-stationary goals to stress-test estimator limits under conditions where LS matrix conditioning fails. Extending the algorithmic suite to evaluate additional non-linear filters, such as the **Unscented Kalman Filter**, alongside **hybrid classical path planners**, represents another logical progression. Finally, incorporating automated **curriculum learning** by sorting training environments by spatial complexity (from open layouts to intricate corridor networks) offers a clear path toward enhancing learning efficiency and policy optimization.

Bibliography

- Arulampalam, M. S., Maskell, S., Gordon, N., & Clapp, T. 2002, IEEE Transactions on Signal Processing, 50, 174
- Bajcsy, R. 1988, Proceedings of the IEEE, 76, 996
- Bajcsy, R., Aloimonos, Y., & Tsotsos, J. K. 2018, Autonomous Robots, 42, 177
- Bar-Shalom, Y., Li, R.-Y., & Kirubarajan, T. 2001, Estimation with Applications to Tracking and Navigation (New York, NY: John Wiley & Sons)
- Foead, D., Ghifari, A., Kusuma, M. B., Hanafiah, N., & Gunawan, E. 2021, Procedia Computer Science, 179, 507, doi: <https://doi.org/10.1016/j.procs.2021.01.034>
- Gadre, A., & Stilwell, D. 2004, in IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA '04. 2004, Vol. 5, 4472–4477 Vol.5, doi: [10.1109/ROBOT.2004.1302422](https://doi.org/10.1109/ROBOT.2004.1302422)
- Gadre, A. S., & Stilwell, D. J. 2005, Proceedings of the 2005, American Control Conference, 2005., 656. <https://api.semanticscholar.org/CorpusID:1494346>
- Han, R., Wang, S., Li, C., et al. 2026, IR-SIM: A Lightweight Skill-Native Simulator for Navigation, Learning, and Benchmarking. <https://arxiv.org/abs/2606.08729>
- Larsen, M. B. 2000, in Proceedings of MTS/IEEE Conference Oceans 2000, Vol. 3, 2043–2050, doi: [10.1109/OCEANS.2000.882243](https://doi.org/10.1109/OCEANS.2000.882243)
- Li, T., Bolić, M., & Djurić, P. M. 2015, IEEE Signal Processing Magazine, 32, 70
- Li, T., Ho, D., Li, C., et al. 2020, in Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 5839–5846, doi: [10.1109/IROS45743.2020.9341284](https://doi.org/10.1109/IROS45743.2020.9341284)
- Li, Y. 2025, Theoretical and Natural Science, 109, 124, doi: [10.54254/2753-8818/2025.GL24049](https://doi.org/10.54254/2753-8818/2025.GL24049)
- Masmitja, I. 2021, RLforUTracking, <https://github.com/imasmitja/RLforUTracking>, GitHub

- Masmitja, I., Gomariz, S., Del-Rio, J., et al. 2019, IEEE Access, 7, 86946, doi: [10.1109/ACCESS.2019.2924722](https://doi.org/10.1109/ACCESS.2019.2924722)
- Masmitja, I., Martin, M., Katija, K., Gomariz, S., & Navarro, J. 2022, in Proceedings of the IEEE 18th International Conference on Automation Science and Engineering (CASE), 675–682, doi: [10.1109/CASE49997.2022.9926499](https://doi.org/10.1109/CASE49997.2022.9926499)
- Mikoff, A. 2020, Particle Filter: localizing the robot, Aleksandr Mikoff’s Blog. <https://mikoff.github.io/posts/particle-filter.md/>
- Murel, J., & Kavlakoglu, E. n.d., Was ist Reinforcement Learning?, <https://www.ibm.com/de-de/think/topics/reinforcement-learning>
- Murphy, K. 2025, Reinforcement Learning: An Overview. <https://arxiv.org/abs/2412.05265>
- Newman, P., & Leonard, J. 2003, in Proceedings of the 2003 IEEE International Conference on Robotics and Automation (ICRA), Vol. 3, 3285–3292, doi: [10.1109/ROBOT.2003.1242111](https://doi.org/10.1109/ROBOT.2003.1242111)
- Raffin, A., Hill, A., Gleave, A., et al. 2021, J. Mach. Learn. Res., 22
- Scherbatyuk, A. P. 1995, in Proceedings of the OCEANS ’95 MTS/IEEE, Vol. 3, 1620–1623, doi: [10.1109/OCEANS.1995.528667](https://doi.org/10.1109/OCEANS.1995.528667)
- Setoodeh, P., Habibi, S., & Haykin, S. 2022, Particle Filter (Wiley), 71–83, doi: [10.1002/9781119078166.ch6](https://doi.org/10.1002/9781119078166.ch6)
- Simon, D. 2006, Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches (Hoboken, NJ: John Wiley & Sons)
- Simonini, T., & The Hugging Face Team. 2022, Deep Reinforcement Learning Course, <https://huggingface.co/learn/deep-rl-course>
- Sipser, M. 2012, Introduction to the Theory of Computation, 3rd edn. (Boston, MA: Cengage Learning)
- Sutton, R. S., & Barto, A. G. 2018, Reinforcement Learning: An Introduction, 2nd edn. (Cambridge, MA: MIT Press)
- Thrun, S., Burgard, W., & Fox, D. 2005, Probabilistic Robotics (Cambridge, MA: MIT Press)
- Tommasi, E. 2026, IRSIM-Range-Only-Beacon-RL, <https://github.com/Maedoster/IRSIM-Range-Only-Beacon-RL>
- Towers, M., Jordan, J. K., Raffin, A., et al. 2024, Journal of Machine Learning Research, 25, 1. <https://github.com/Farama-Foundation/Gymnasium>
- Vaganay, J., Leonard, J. J., Curcio, J. A., & Willcox, J. S. 2004, Virtual Long Baseline (VLBL) Autonomous Underwater Vehicle Navigation Using a Single Transponder, Tech. rep., MIT Sea Grant College Program

Acknowledgements

I want to express my deepest gratitude to everyone who has supported me on this journey to reach this significant milestone in my life.

Thank you to the University of Bologna and the University of Bielefeld for providing safe and inspiring environments that encouraged me to experiment with subjects I never imagined I would study. I am immensely grateful for the Erasmus+ program and the Double Degree opportunity, which opened the doors to this international journey, broadened my horizons, and allowed me to grow both as a scholar and as a person.

I am sincerely grateful to my second reviewers, PD Dr.-Ing. Sven Wachsmuth and Prof. Simone Martini (Bologna), for their guidance with both my thesis and daily life in Bielefeld, offering valuable information whenever I needed it most.

My utmost gratitude goes to my first reviewer, RA Jesus E. Aleman G., for his immense help in revising this thesis and guiding me as I built this entire project from scratch. Without his dedication, this work would have taken many more months and would not have reached this level of quality. *'La Dolce Vita'* has never sounded better than this.

Thank you to all my friends here in Bielefeld, who made my days so much lighter and my study sessions far less miserable. It takes courage to be vulnerable, to explore the world, and to trust that our choices and dreams can come true. Still, we pace on; we wander the unknown to prove that our worth is not imaginary, but a fire that burns.

Michele, Vittorio, Cono, Giulio, Andreea, Luca, Gabriele, Benedetta, Sofia, Enrico, Ginevra, Elena, Claudia, Majo, Regina, Fanny, Fito, Miguel, Kevin, Atahualpa, Yahiza, Tommaso, Ave, JuangMi, Marcos, Yago, Sofia, Sara, Rachele, Angelica, Pedro, Vaso, Andrea, Joanna, the whole RoboCup Team, and the EDG theatre group of Bielefeld.

Thank you to all my childhood friends and those I grew up with. Your company and solace allowed me to believe in myself when I needed it most. You taught me that being authentic is the most precious thing in this world, and that our souls are vast and deserve to be known.

Valerio, Gaia, Sara, Filippo, Alberto, Francesca, Alessandro, Luca, Eduard, Rosatea, Daniel, Chiara, Leonardo, Ana, Francesca, Luca, Enrico, Pasquale, Giovanni, Daniele, Eliana, Paolo, Chiara, Katiuscia, Leonardo, Francesco, Andrea, Samuel, Federico, Lucrezia, Luna.

Thank you to my family for believing in me my entire life, for raising me, and for showing me that the world can be a warm place. Though it can sometimes be hard to understand each other and easy to quarrel, I will love you all until the end of my life.

Alessandra, Chiarina, Marcello, Isabella, Dario, Marco, Donatella.

Thank you to the little Edoardo, the not-so-little Edoardo, and the grown-up Edoardo for choosing to live despite it all. It's okay to be fearful; you are enough just the way you are.

Every single one of these names is proof that leaving a mark on this world isn't about grand gestures, it is about sharing love and understanding that vulnerability is never a weakness. Real weakness comes from a soul that fears being loved. Be passionate, yearn for life, dance like a falling star, **Be**.

*Shining through the hardships of life
requires the heart of a **lion**
and the tenderness of a crying **child**.*